轮 趣 科 技

避障巡线雷达小车
L150 用户手册

推荐关注我们的公众号获取更新资料

版本说明：

版本	日期	内容说明
V1.0	2022/07/01	第一次发布（初稿）
V1.1	2022/07/13	内容完善与补充
V2.0	2023/03/02	内容部分修改和补充
V2.1	2023/06/01	优化 ROS 接口教程内容
V2.2	2024/3/25	优化雷达部分教程内容
V2.3	2024/4/19	增加航模控制教程内容
V2.4	2025/12/17	增加 LD14P 雷达开始使用说明

序言

本教程的内容主要针对避障巡线智能小车 L150 的使用教程。避障巡线智能小车使用 F103RCT6 主控芯片，外设丰富，性能强大。主控板不仅可用于小车的控制，还能用作开发板进行学习，兼容大部分野火电子的教程；程序源码容易阅读，修改简单，可移植性强。

手册中涵盖使用过程中出现的大部分问题，若遇到问题，请先仔细阅读手册。

目录

序言 2

1. 小车快速上手使用步骤与注意事项 5

1.1 小车主控介绍 5

1.2 使用步骤 5

1.3 速度控制单位的计算与说明 7

2. 功能使用教程 8

2.1 APP 遥控 8

2.2 PS2 无线手柄遥控 11

2.3 航模控制 12

2.4 雷达避障 13

2.5 雷达跟随 14

2.6 雷达走直线模式 14

2.7 电磁巡线 15

2.8 CCD 巡线 15

2.9 上位机波形查看 16

3. 代码框架与运行逻辑说明 17

3.1 程序文件目录和程序框架说明 17

3.2 电机控制 19

3.3 雷达避障模式控制逻辑 19

3.4 雷达跟随模式控制逻辑 21

3.5 雷达走直线模式控制逻辑 21

4. 运动学分析 23

4.1 两轮差速(履带车)小车 23

4.2 阿克曼小车 26

5. 常用参数修改 35

5.1 雷达避障参数修改 35

5.2 遥控速度参数修改 36
5.3 雷达跟随参数修改 37
5.4 雷达走直线参数修改 37
5.5 电机或轮子更换后参数调整 37
5.6 扩展部分 38
6. ROS 接口 41
6.1 ROS 接口控制输入协议说明 41
6.2 ROS 接口默认发送的数据协议 42
7. 使用注意事项 48
7.1 关于小车使能开关 48
7.2 关于电磁传感器 48
7.3 关于电机 48
7.4 关于电池 48
8. 程序下载 49
8.1 串口一键下载 49
8.2 ST-Link 下载 50

1. 小车快速上手使用步骤与注意事项

1.1 小车主控介绍

小车由运动底盘、F103RCT6 主控（下文简称为主控）、传感器模块组合而成。小车主控集成了所有功能的接口，具体请看图 1-1-1。

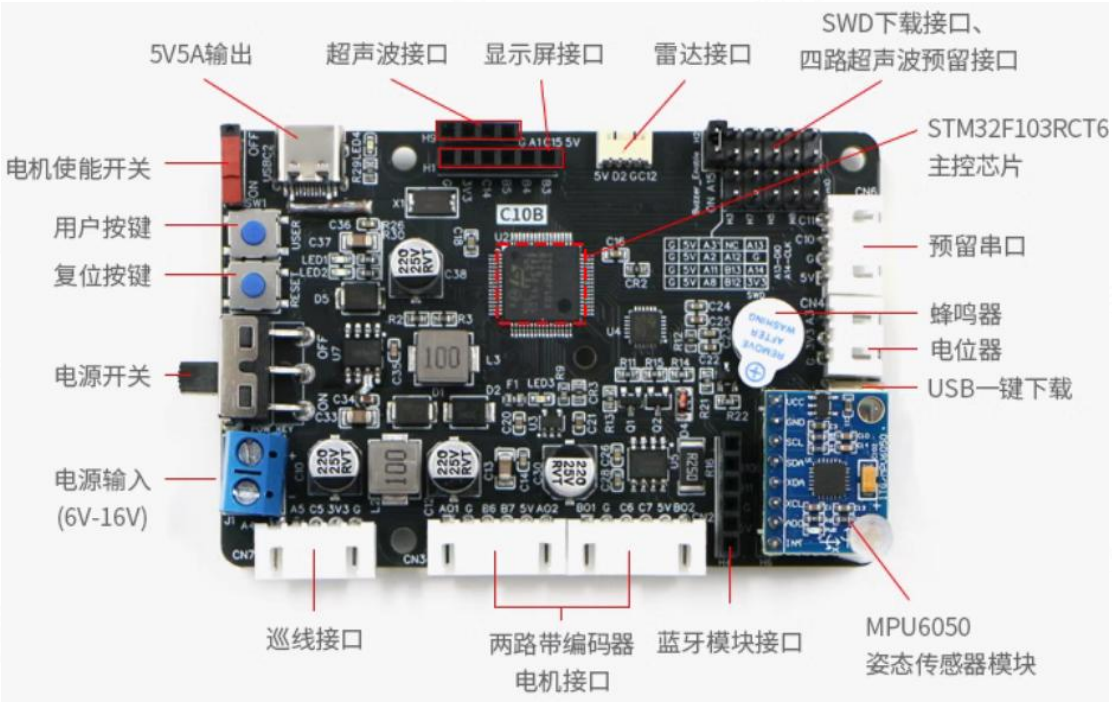


图 1-1-1 主控详细说明

默认为外接模式，需要的用户可自行扩展。详情请参考“5.6 拓展部分的陀螺仪部分说明”。

1.2 使用步骤

① 小车开机上电与车型选择

小车为整车发货，用户到手即可上电使用，上电仅需拨动主控左边的电源开关。上电后显示屏会亮起，显示的是正常的运动模式，此时旋动主控右下角的电位器可以选中对应的车型（旋转电位器时显示屏上第六行的左边车型会变化），复位后即可生效。该主控支持 4 种不同的运动底盘：包括：差速（Diff Car）、阿克曼（Akm Car）、标准履带（S Tank Car）、高配履带四款车型（B Tank Car）。具体操作步骤参考图 1-2-2。

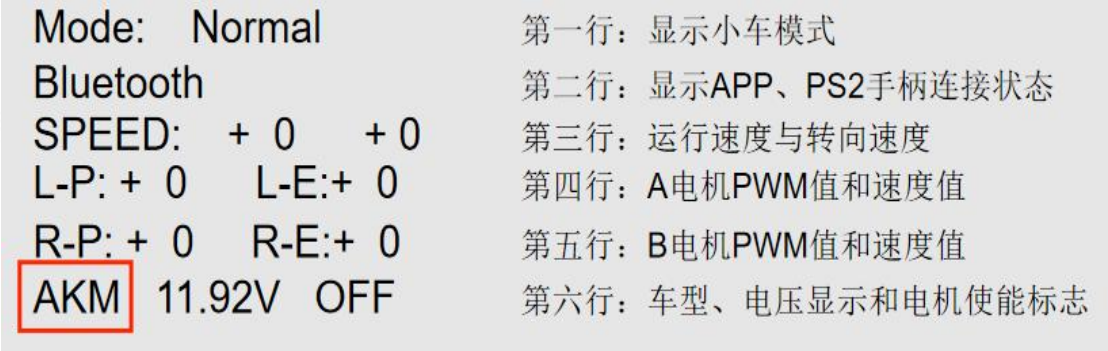


图 1-2-1 车型选择显示

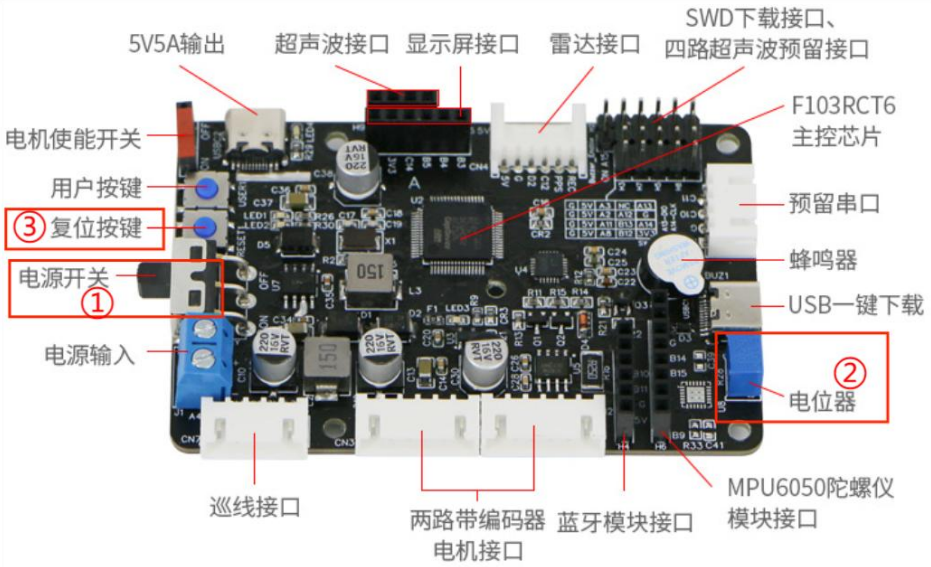


图 1-2-2 操作步骤

② 功能选择

小车带有：APP 遥控（Normal）、无线手柄遥控（Normal）、航模控制（Normal）、雷达避障（Avoid）、雷达跟随（Follow）、雷达走直线（Along）、CCD 巡线（CCD）、电磁巡线（ELE）的功能，通过单击“用户按键”进行切换，开机默认为 Normal 模式。

其中 Normal 模式包含 APP 遥控、无线手柄遥控和航模遥控功能，整车默认标配 APP 蓝牙模块；无线手柄遥控功能需要搭配无线手柄和无线手柄接收器才可以使用；航模遥控功能需要搭配航模遥控器和航模接收器才可以正常使用；雷达避障（Follow）模式、雷达跟随（Follow）模式、雷达走直线（Along）模式需要搭载乐动 LD14/LD14P 激光雷达（自 2025 年 12 月 17 日开始使用 LD14P 雷达，两款雷达烧录对应的代码均可正常使用）才可以使用；CCD 巡线（CCD）功能需要搭载 CCD 功能模块才可以使用；电磁巡线（ELE）功能需要搭载电磁巡线模块

与电磁线才可以使用；电磁巡线模式可以搭配乐动 LD14/LD14P 激光雷达实现巡线过程的遇障停车功能。具体每个功能的使用请看下一章“2.功能使用教程”。

1.3 速度控制单位的计算与说明

小车的速度控制主要是控制 X 轴（前后方向）和 Z 轴（旋转方向）的速度，以 Vx 和 Vz 来指代，单位分别是 m/s 和弧度/s。X 轴方向以前进记为正，Z 轴方向以右转记为正。

车轮速度是使用编码器来计算和得出，读取编码器计数后再转化成车轮的速度。在 control.c 文件中的 void Get_Velocity_From_Encoder(void)函数有编码器转换成速度的公式，至于 Vz，阿克曼车型代表的是转向角度，单位是 rad；非阿克曼车型都是需要用到左右轮的速度进行计算，具体查看 4.运动学分析 这一章节。

2. 功能使用教程

请注意，任何的控制模式都会受到使能开关的控制，使能开关转态为“ON”时电机才可以受控制，使能开关状态可以在显示屏的右下角看到，使能开关在主控的左上角位置。

2.1 APP 遥控

小车进入控制程序后，选中 Normal 模式。蓝牙模块未连接时，模块上的红灯是慢闪状态，打开手机的蓝牙设置界面，找到名称为 BT04A 设备进行配对，配对密码为 1234。配对成功后打开 mini_balance 的手机 APP，在设备界面选中 BT04A 设备即可连接并实现控制，成功连接后 Debug 界面会一直刷新数据。

此时若要控制小车，需要上拉摇杆 1s 左右，小车的显示屏第二行的字会由“LINKING...”变为“Bluetooth”，此时就可以控制小车了。



图 2-1-1 蓝牙连接后界面

APP 操作界面对应的每一个操作是实际上向小车发送不同的指令信息（切换控制方式及界面也同样发送指令），机器人接收到指令信息后作出响应，详细的 APP 操作界面的每个操作对应发送的信息见表 2-1-1。

表 2-1-1 APP 界面操作指令说明

APP 摇杆	↑	↗	→	↘	↓	↙	←	↖
小车接收到的数据	0x41	0x42	0x43	0x44	0x45	0x46	0x47	0x48
小车实现效果	前进运动	右前运动	原地右转	右后运动	后退运动	左后运动	原地左转	左前运动
按键	重力		摇杆		按键		减速	加速
小车接收到的数据	0x49		0X4a		0x4b		0x59	0x58

APP 不仅能遥控小车，还能通过 APP 监控小车，并进行参数的修改。在“首页”可以看到小车的左右轮的速度(mm/s)和电池电压等信息；在“波形”界面可以看到小车状态的变化，目前设定两个波形，分别是左右编码器的读数；在“调试”界面可进行参数调整，目前设定参数 0 和参数 1 分别是蓝牙遥控的速度和 CCD 巡线的速度。上述内容的具体信息都可以通过代码进行设定，在 show.c 文件中的 APP_Show()函数和 bluetooth.c 文件中的蓝牙中断函数 BLUETOOTH_USART_IRQHandler()进行修改即可。

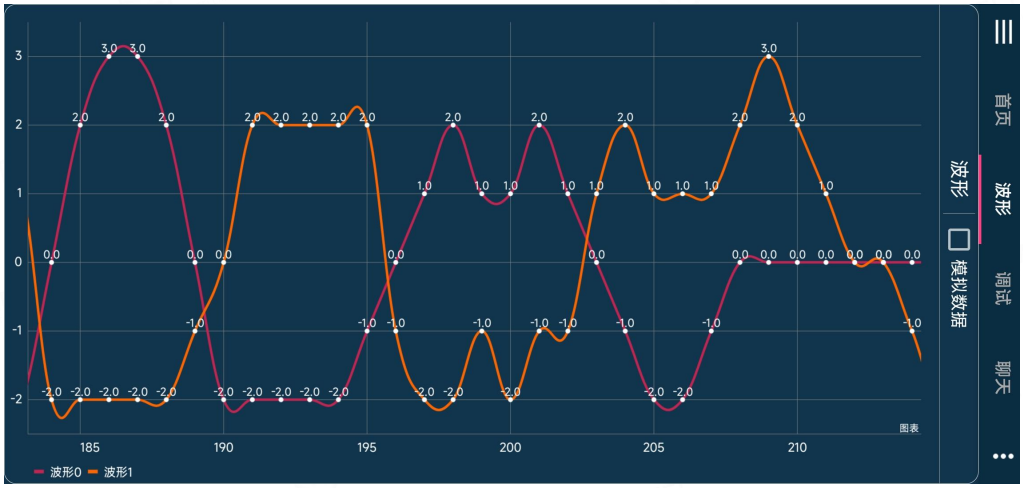


图 2-1-1 波形界面



图 2-1-2 参数界面

调参界面可以修改参数的名称，只需点击参数的名称即可弹出设置；可自定义按键发送的内容，长按下面的自定义按键，即可弹出设置。



图 2-1-3 参数设置界面



图 2-1-4 自定义按键界面

2.2 PS2 无线手柄遥控

小车进入控制程序后，选中 Normal 模式，然后打开手柄的开关，如果配对成功，手柄接收器的灯会常亮，不成功则闪烁。观察红色的模式指示灯是否亮起，如果没有亮起，需要按下手柄的“MODE”按键，此时会进入控制的模式。无线手柄按下“START”按键，然后向上推动左边手柄摇杆，即可控制小车。



图 2-2-1 手柄连接界面

手柄左边的摇杆的上下方向操控小车的前进和后退，右边的摇杆的左右方向操控小车的转向，幅度越大代表速度越大；手柄上边的左 1 键可以进行加速，右 1 键能进行减速。



图 2-2-2 手柄控制图示

2.3 航模控制

首先连接小车主板和航模接收器，L150 小车使用了航模的通道一和通道二进行控制，通道 1 连接主板的 A2 引脚，通道 2 连接主板的 A3 引脚，该部分引脚在主板的右上方的预留 IO 排针处。

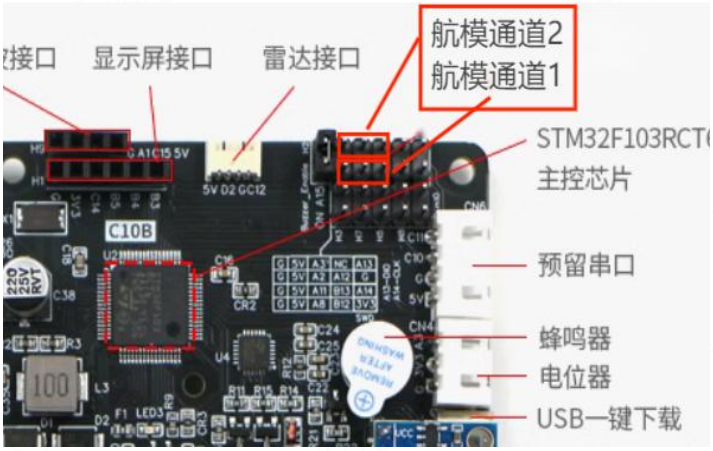




图 2-3-1 航模接收器连接处图示

小车进入控制程序后，选中 Normal 模式，然后打开航模的开关，如果配对成功，航模接收器的灯会常亮，不成功则闪烁。连接成功后向上推动左边手柄摇杆，即可控制小车，航模左边摇杆前后推动控制小车前进和后退，右边摇杆的左右推动控制小车的左右转动。



图 2-3-1 航模控制模式显示屏界面



图 2-3-2 航模图示

2.4 雷达避障

小车进入控制程序后，选中雷达避障 Avoid 模式。小车会有一个初始的向前速度，如果前方遇到障碍物时，会自转避开障碍物。暂时不支持小车在遥控过程自动避障的功能。

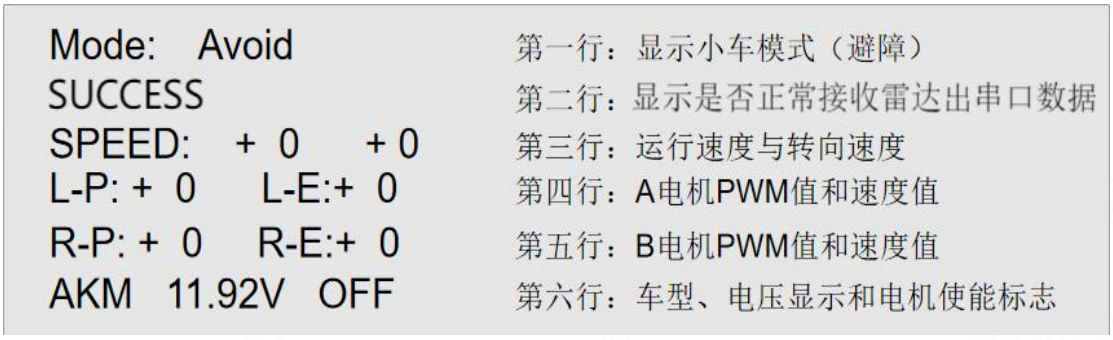


图 2-4-1 雷达避障选定界面

2.5 雷达跟随

小车进入控制程序后，选中雷达跟随（Follow）模式。小车会开始运动，在360°范围内自动跟随距离小车最近的物体。但是小车并不会直接撞上距离最近的物体，而是与最近物体保持一定的距离，这个距离值为400mm。若小车与最近的物体距离超过这个保持的距离值，小车会跟过去，与最近物体距离小于这个保持距离值小车会后退。

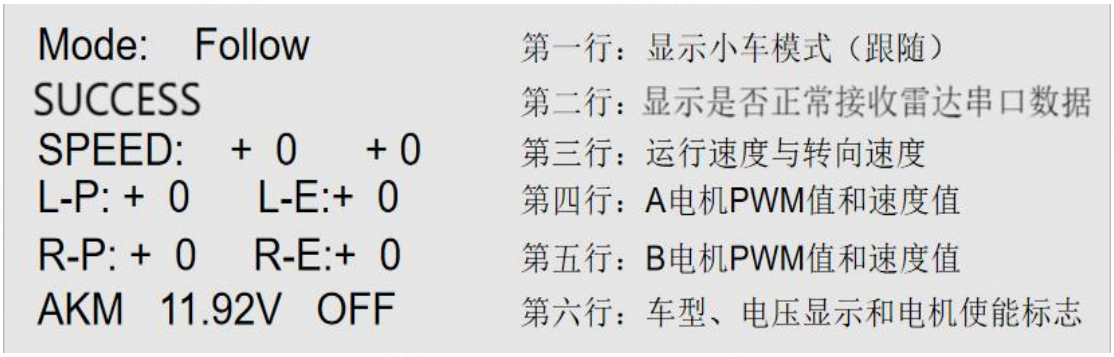


图 2-5-1 雷达跟随选定界面

2.6 雷达走直线模式

小车打开电源开关后，这个模式是依靠根据对照物体来实现的，先将使能开关打到 off，然后将小车与参照物平行摆放，接着切换到走直线模式（Along），然后打开使能开关小车开始前进走直线。小车就可以根据这个直面物体开始走直线，我们设定的是小车依据右方的物体走直线，需要改动左右方向，可参考下面5.3 小节雷达走直线参数修改。



图 2-6-1 雷达走直线模式

2.7 电磁巡线

小车进入控制程序后，选中电磁巡线（ELE）模式。电磁巡线模式可以自主选择是否接入激光雷达的数据进行停车避障，在此模式中双击用户按键即可切换。



图 2-7-1 电磁巡线模式显示屏界面

电磁巡线需要使用通电的漆包线作为巡线条件，具体原理可以参考“电磁巡线原理手册”。将小车放在电磁巡线赛道中观察显示屏的第 1 行是否有读数，且前三个读数相差较大，第四个读数较为稳定则为正常工作状态。前三个读数表示左中右三个感应元件的读数，第四个读数是用算法综合三个感应元件的读数得出的数值，用于指示漆包线的位置，若线正好在中间，那么读数应为 100。

程序的第五行显示是否开启接入激光雷达数据，双击用户按键可以进行切换，显示为“ON”表示接入激光雷达数据，显示“OFF”表示不接入雷达数据。在开启激光雷达数据接入模式时，小车在巡线过程中遇到障碍物会停车，并有蜂鸣器响声，直到障碍物消除后才会继续巡线。默认障碍物判断距离为：70cm。

2.8 CCD 巡线

将 CCD 传感器通过排线与小车主控上的接口进行连接。小车进入控制程序

后，选中 CCD 巡线（CCD）模式。

CCD 巡线模式由于机械安装干涉的缘故，所以不支持融合激光雷达进行停车避障。将小车放在 CCD 巡线赛道中，可以见到显示屏最上端会有一个滑块，滑块为 CCD 传感器识别到的赛道，小车在赛道中可以自动巡线。



图 2-8-1 CCD 巡线模式显示屏界面

2.9 上位机波形查看

我们提供了上位机示波器软件，可以进行数据的观察。首先通过 USB 线将主板与 windows 电脑连接，俺后长按主板的“用户按键”2s，松手后小车会以波特率 115200 向 PC 端发送数据包，因为上位机需要严格的时序，使用上位机的时候，小车会停止 OLED 的刷新，并不再向 MiniBalance APP 发送数据。

注意，上位机软件不能双击打开，必须右键以管理员模式运行。波特率设置为 115200，端口号需要通过“设备管理器”查看主板对应为哪个 COM 口。目前上位机设定的波形有三个，分别是左右轮的速度(mm/s)和电池电压，需要显示其他波形，在 show.c 文件中的 DataScope()函数进行更改即可。

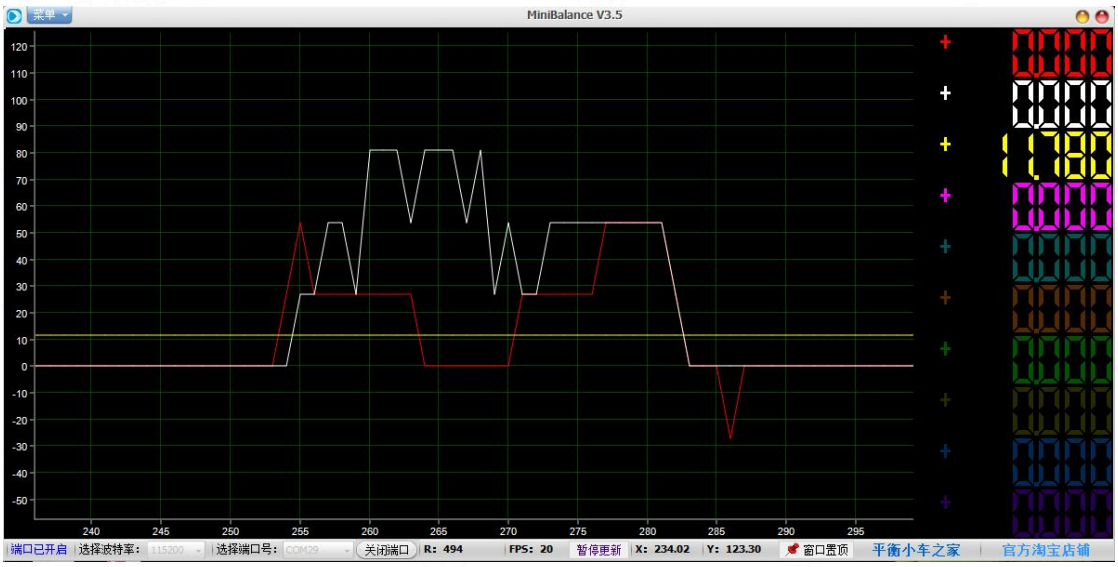


图 2-9-1 上位机波形图示

3.代码框架与运行逻辑说明

3.1 程序文件目录和程序框架说明

① 程序的目录结构

程序目录主要包含 stm32 相关启动文件和库函数，硬件相关初始化代码和控制逻辑部分代码。目录如下：

```
MiniBalance:
 ├──STARTUP
 |   └──startup_stm32f10x_hd.s       //stm32 启动文件
 |
 ├──CMSIS                            //Cortex 微处理器接口
 |   ├──core_cm3.c
 |   ├──core_cm3.h
 |   └──......
 |
 ├──FWIB                             //stm32 标准库文件
 |   ├──misc.c
 |   ├──stm32f10x_adc.c
 |   ├──stm32f10x_bkp.c
 |   └──......
 |
 ├──USER                             //主要是用户文件，main 函数入口
 |   ├──main.c                       //main 函数入口
 |   ├──stm32f10x_conf.h             //配置文件等
 |   ├──stm32f10x_it.c
 |   ├──stm32f10x_it.h
 |   └──Header.h                     //公共头文件
 |
 ├──HARDWARE                         //主要是硬件部分的代码
 |   ├──bsp_beep.c
 |   ├──bsp_key.c
 |   ├──bsp_led.c
 |   ├──bsp_SysTick.c
 |   └──......
 |
 ├──CONTROL                          //控制逻辑部分代码
 |   ├──control.c                    //控制函数在这，5ms 中断
 |   ├──show.c                       //显示屏函数，app 显示函数和上位机示波器
 |   ├──bluetooth.c                  //蓝牙部分控制代码
 |   ├──pid.c                        //pid 反馈
 |   ├──Lidar.c                      //雷达串口接口和处理代码
```

```
|   ├──capture.c                    //航模和超声波，默认使用航模
|   └──ELE_CCD.c                    //电磁巡线和 CCD 巡线
|
├──MPU6050                          //MPU6050
|   ├──inv_mpu.c                    //MPU6050 相关库文件
|   ├──inv_mpu_dmp_motion_driver.c  //MPU6050 相关库文件
|   ├──IOI2C.c                      //I2C 驱动文件
|   ├──MPU6050.c                    //MPU6050 驱动文件
|   ├──filter.c                     //滤波文件，卡尔曼和互补滤波
|   └──MPU6050_Test.c               //MPU6050 测试程序
|
└──DOC                              //文档说明
   └──readme.txt
```

在小车的程序中，重点关注 main.c 和 control.c 文件。main.c 是 main 函数的入口文件，整个程序从 main 函数中开始执行，辅以 5ms 一次的中断，即 control.c 文件中的 TIMING_TIM_IRQHandler()中断函数，让整个小车程序得以运行。

② **程序整体框架**

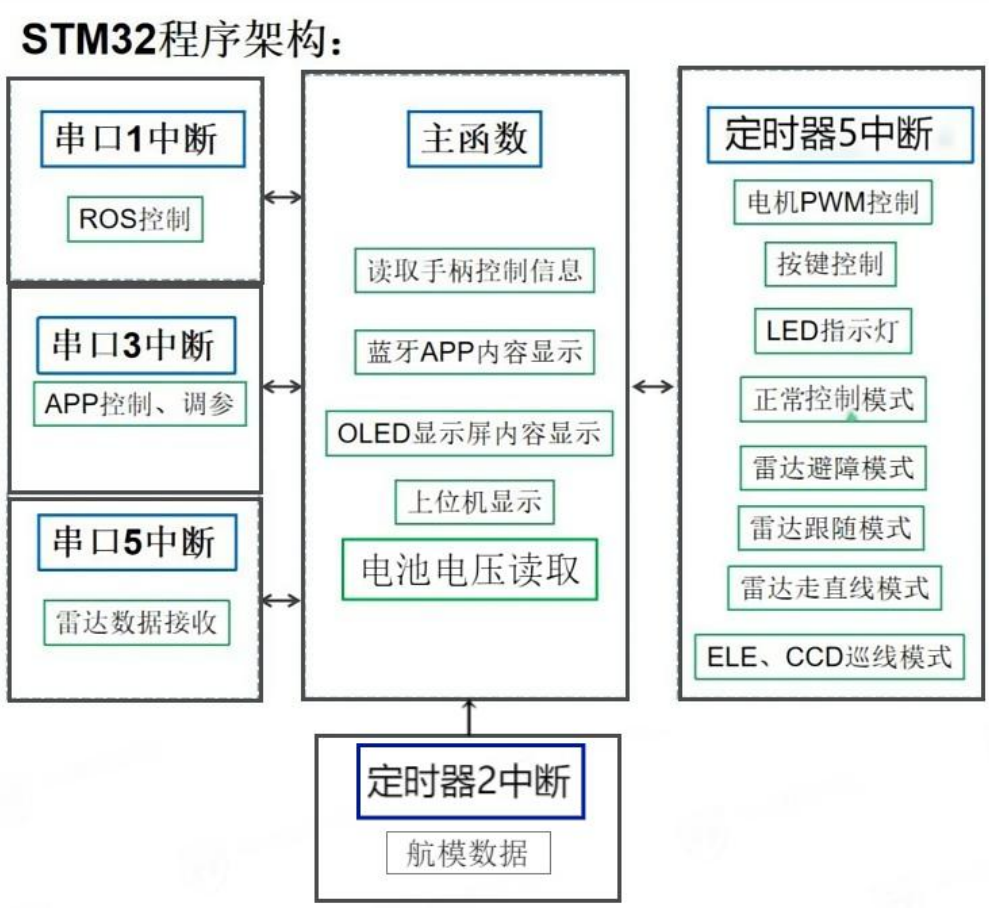


图 3-1-1 STM32 程序架构

3.2 电机控制

小车电机驱动器芯片使用 AT8236，具有过流保护功能，并可设置电流阈值。驱动芯片只需两个逻辑输入，便可达到调速和正反转的功能，本小车中，每个电机使用两路 PWM 进行调速（实际上一个普通 IO 和一路 PWM 即可进行正反转和调速）。

电机的速度使用 13 线霍尔编码器输出 AB 相进行测量，电机减速比为 1:30，使用 STM32 的编码器测量功能，并初始化为脉冲上升沿和下降沿都进行计数，可实现轮子转一圈输出 1560 个计数。

电机调速框图如图 3-2-1 所示，其中舵机部分仅阿克曼车型有。

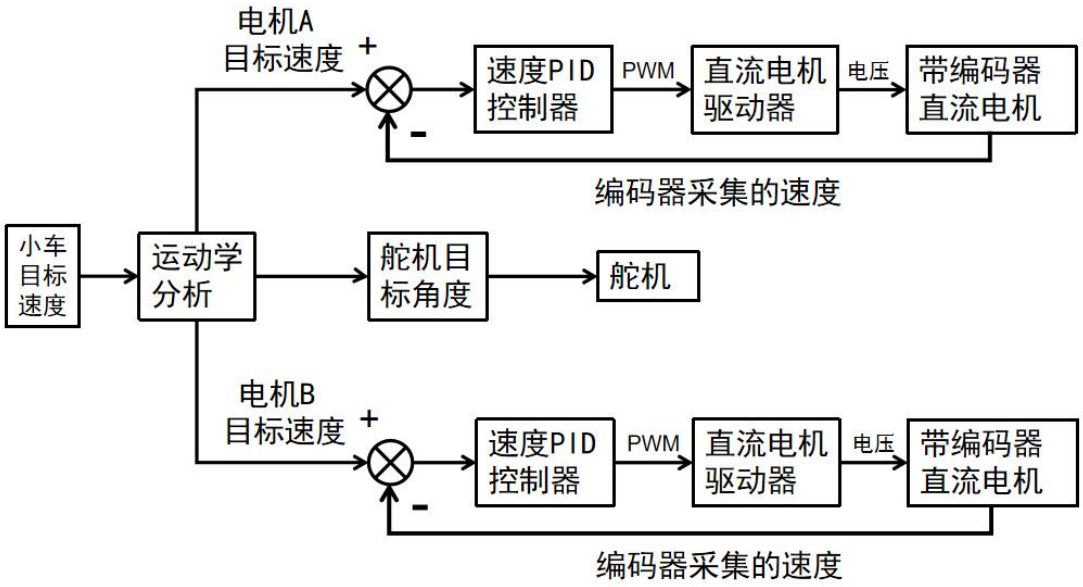


图 3-2-1 电机调速框图

3.3 雷达避障模式控制逻辑

雷达避障和雷达跟随功能需要使用到雷达检测到的数据，雷达使用串口通信，在上电的时候会自动传回数据包，不需要进行额外的设置，此时只需要在串口中断中进行接收即可。但是我们知道，串口通信每次只接收一个八位的数据，如何把整个一帧数据存储下来呢，可以使用一个状态变量来标志当前的数据包的位置，通过结构体或数组之类的变量储存下来。处理流程如图 3-3-1 所示。

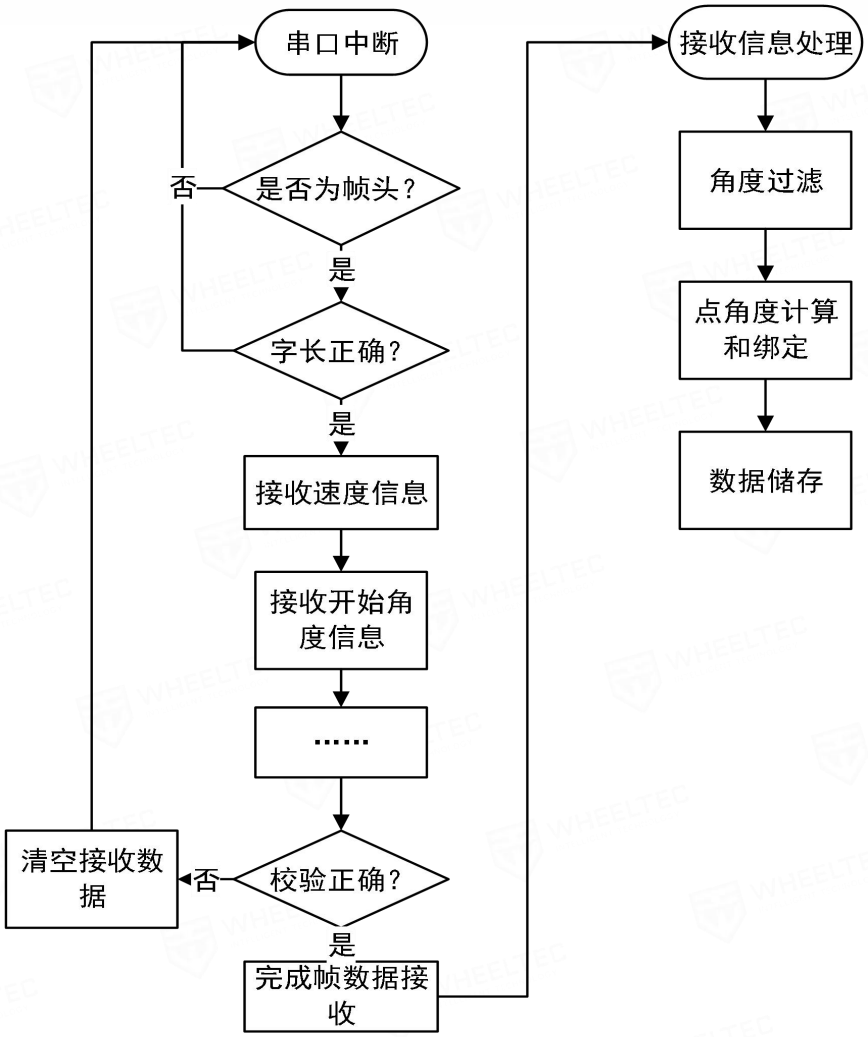


图 3-3-1 雷达数据接收流程

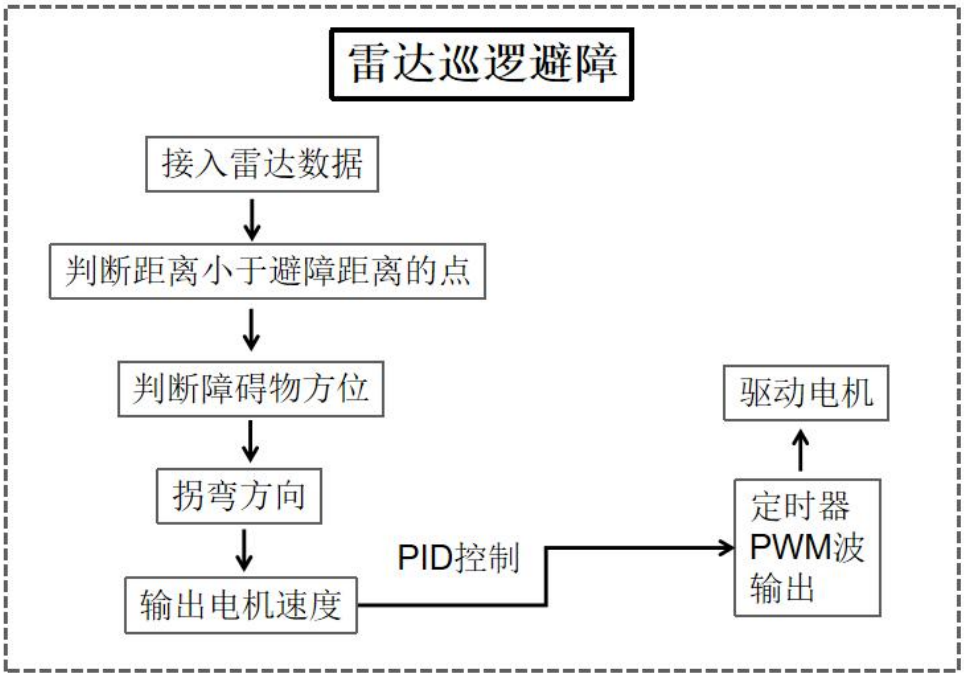


图 3-3-2 雷达避障逻辑框图

小车进行避障的时候，会有一个向前的速度，并且实时调用雷达的数据来分辨前方是否有障碍物，若无障碍物则继续前进，若有障碍物则进行判断障碍物的位置，如果障碍物在左边小车则右转，如果障碍物在右边则左转，直到前方检测不到障碍物为止。框图如图 3-3-2 所示。

3.4 雷达跟随模式控制逻辑

雷达跟随模式能够实现全方向地跟随，并且可以和物体保持一定的距离。雷达传输回来的数据不仅有距离，还有帧数据的起始角度和结束角度，雷达跟随模式就用到了这两个信息。这个模式主要用到两个 PID 控制器，一个是距离 PID，控制小车和被跟随物体之间的距离，另一个是角度调整 PID，控制小车车头的方向，保证了小车永远对着被跟随的物体。框图如图 3-4-1 所示。

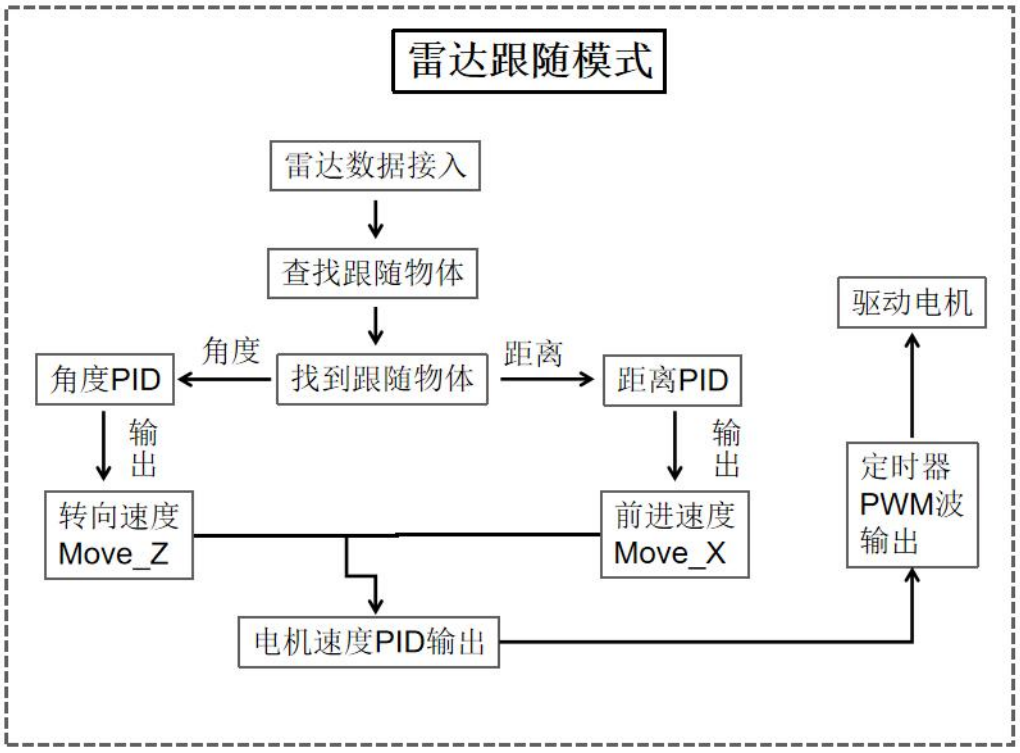


图 3-4-1 雷达跟随模式框图

3.5雷达走直线模式控制逻辑

雷达走直线模式是根据参照物来运行的，当然物体多远都行，只要在雷达探测的范围，我们默认设定了车前进方向右侧的走直线模式，想要换方向只需要改

下接收雷达探测的数据角度就行。这个模式主要用到距离调节 PID，努力保证小车行驶在距离参照物的目标距离内。框图如图 3-5-1 所示。

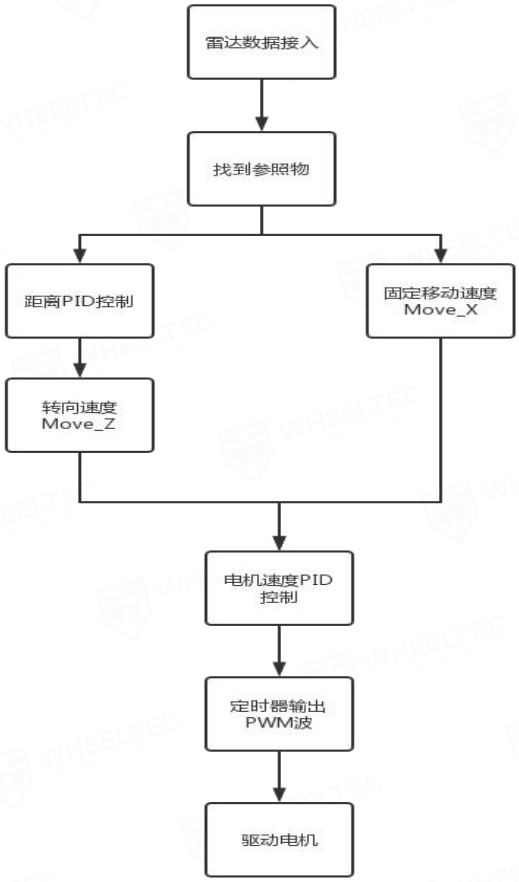


图 3-5-1 雷达走直线模式框图

4. 运动学分析

所有的车型共用一个代码，因此不同的车型会对应不同的运动学分析解算。

4.1 两轮差速(履带车)小车

① 运动学分析

两轮差速式机器人是比较常见的轮式机器人，它控制最为简单。两轮指的是它只有两个驱动轮，它的特点是从动轮为全向轮或者万向轮等可以进行全向移动的轮子，因此只需要控制两个驱动轮的速度存在差异，即两轮差速，即可控制机器人实现无滑动摩擦的旋转，也可以实现零半径转弯。下图 4-1-1、4-1-2 分别为全向轮、万向轮两轮差速式机器人，两者相比较，全向轮的特点是控制比较稳定，但是负载能力相对较弱，万向轮的控制不稳定体现在急转向的时候会有一定的甩尾效应。

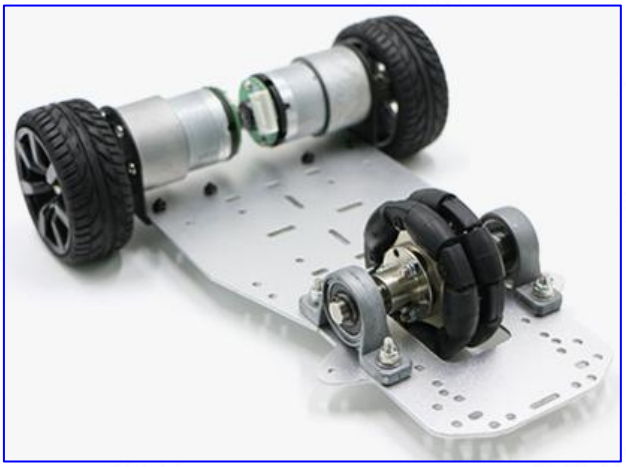


图 4-1-1 全向轮两轮差速式机器人

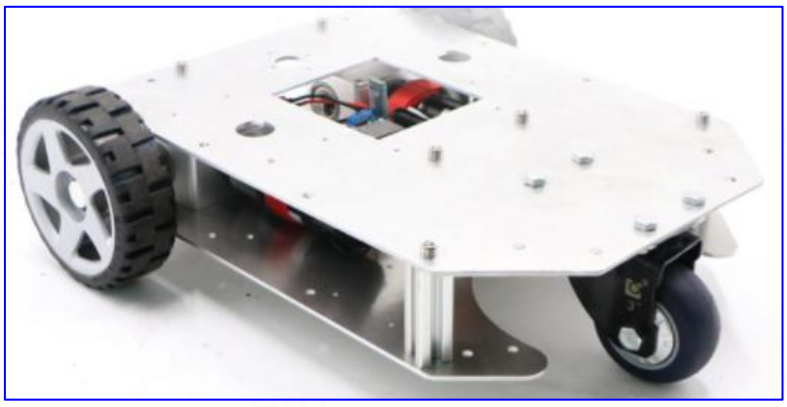


图 4-1-2 万向轮两轮差速式机器人

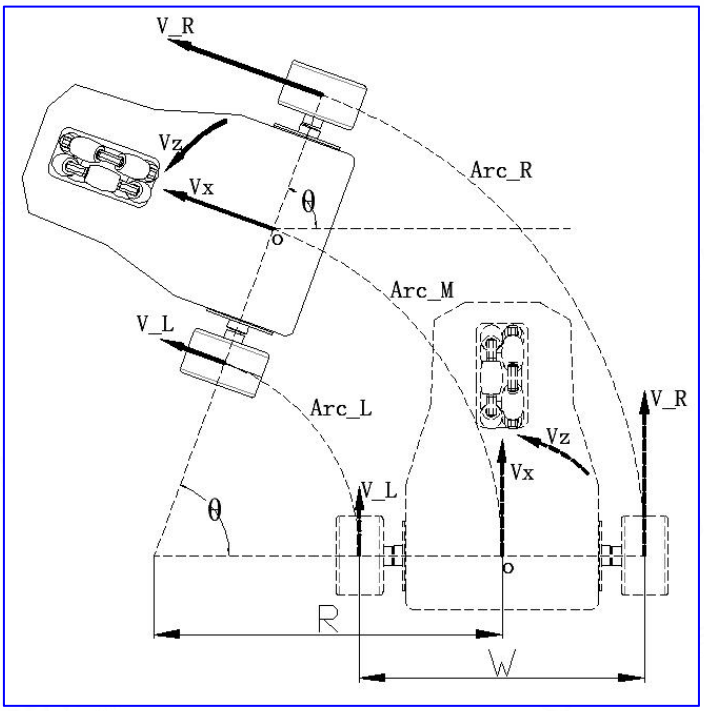


图 4-1-3 两轮差速式机器人运动学分析图

下面我们对两轮差速式机器人进行运动学分析，分析图如上图 4-1-3 所示，其中参数分别代表：

W：两个驱动轮的距离，距离中心为 O 点，单位：m。

V_x：机器人在 O 点的目标前进速度，前进为正，单位：m/s。

V_z：机器人绕 O 点的目标旋转速度，逆时针为正，单位：rad/s。

R：机器人同时前进和旋转产生的转弯半径 R，单位：m。

V_L、V_R：机器人左右轮速度，配合实现目标速度V_x、V_z，前进为正，单位：m/s。

Arc_L、Arc_M、Arc_R：机器人左轮、O 点、右轮在一定时间 t 内走过的路径，单位：m。

θ：机器人在一定时间 t 内旋转的角度，单位：rad。

下面讨论它们之间的关系，并求出两轮差速式机器人的运动学正逆解公式。

由速度对时间的积分等于路程得：

$$Arc_L = V_L * t，\ Arc_M = V_x * t，\ Arc_R = V_R * t \quad 式(4.1.1)$$

由弧长除以半径等于弧度得：

$$\theta = \frac{Arc_L}{R - \frac{W}{2}} = \frac{Arc_M}{R} = \frac{Arc_R}{R + \frac{W}{2}}$$

$$\theta = \frac{V_L * t}{R - \frac{W}{2}} = \frac{V_x * t}{R} = \frac{V_R * t}{R + \frac{W}{2}} \quad 式(4.1.2)$$

公式两边同时除以 t，即对时间积分得：

$$V_z = \frac{V_L}{R - \frac{W}{2}} = \frac{V_x}{R} = \frac{V_R}{R + \frac{W}{2}} \quad 式(4.1.3)$$

对式(4.1.3)分解可得：

$$R = \frac{V_x}{V_z} \quad 式(4.1.4)$$

$$\frac{V_x}{R} = \frac{V_L}{R - \frac{W}{2}} \quad 式(4.1.5)$$

$$\frac{V_x}{R} = \frac{V_R}{R + \frac{W}{2}} \quad 式(4.1.6)$$

将式(4.1.4)代入式(4.1.5)、式(4.1.6)可得运动学逆解公式，已知目标速度V_x、V_z求驱动轮的目标速度V_L、V_R：

$$V_L = V_x - \frac{W}{2} * V_z \quad 式(4.1.7)$$

$$V_R = V_x + \frac{W}{2} * V_z \quad 式(4.1.8)$$

式(4.1.7)、式(4.1.8)可以变形为运动学正解公式，已知驱动轮的当前速度V_L、V_R求当前机器人的实时速度V_x、V_z：

$$V_x = \frac{V_L + V_R}{2} \quad 式(4.1.9)$$

$$V_Z = \frac{V_R - V_L}{W} \quad 式(4.1.10)$$

② C 语言实现

机器人上面有 2 个带编码器的电机，我们需要通过 C 语言写出上面的运动关系，然后对电机进行控制。代码如下（进行了一定程度的精简）：

```
void Get_Target_Encoder(float Vx,float Vz)
{
    //以下计算左右轮的速度
    MotorA.Target_Encoder = Vx + Vz*Wheelspacing/2.0f;
    MotorB.Target_Encoder = Vx - Vz*Wheelspacing/2.0f;
}
```

注意：以上代码中，Vx 和 Vz 的单位分别是 m/s 和°/rad（弧度制），其中 Vz 是以顺时针为正，而上述分析上是逆时针为正，故此处与上面分析并不矛盾。Wheelspacing 是两个车轮的直线距离。

4.2 阿克曼小车

阿克曼转向式机器人其实就是现代汽车的转向结构，阿克曼式与两轮差速式类似，同样依靠驱动轮的差速实现转弯，但是转弯的同时还需要控制前轮的转角进行配合，否则前轮与地面的摩擦力将会非常大，严重影响机器人转向运动已经磨损轮子。阿克曼转向式的优点是可以使用普通轮子，不需要使用全向轮，可以提高机器人整体强度和减低成本，缺点则是受限于前轮的转角幅度，无法进行 0 半径转弯。如下图 4-2-1 所示，为阿克曼式机器人转向时的状态，要求 4 个轮子运动方向的垂线相交于一点，即 4 个轮子围绕同一个圆心进行旋转。

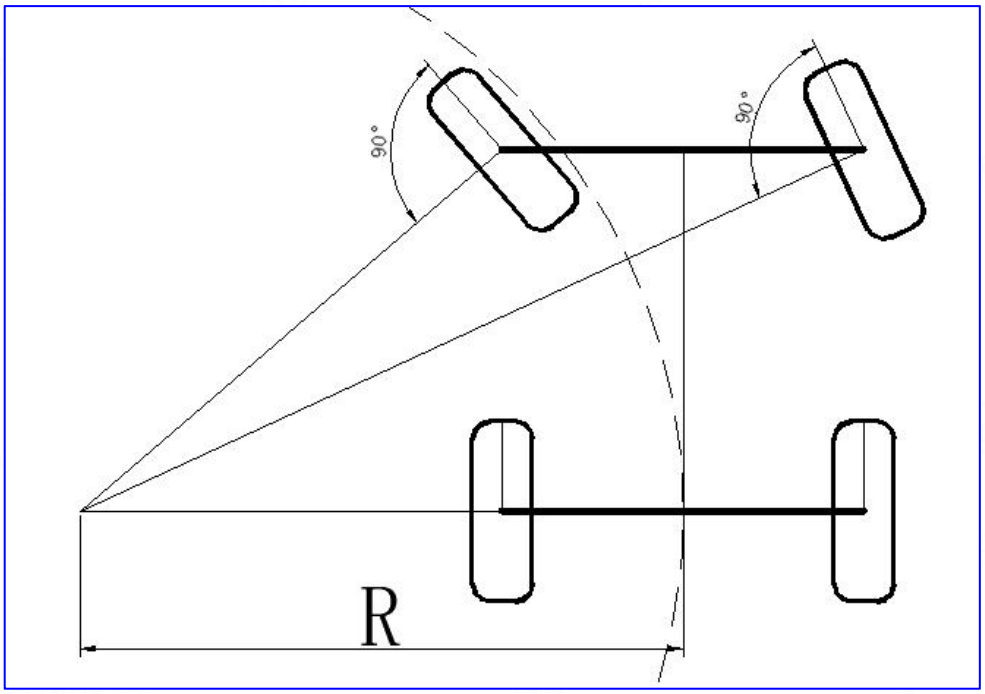


图 4-2-1 典型阿克曼转向状态

① 运动学分析

我们首先讨论运动学逆解，即由 Vx、Vz、Vy 三轴目标速度，求出驱动轮的 目标速度，驱动轮为两个后轮，其速度分别用 V_L 和 V_R 表示，前进为正，后退为负；阿克曼式机器人还要求出两个前轮的偏角，左前轮偏角 AngleL 和右前轮偏角 AngleR，左偏为正，右偏为负。阿克曼式机器人无法直接横向移动，所以不讨论 Y 轴目标速度。

对于阿克曼式机器人，要求运动学逆解，必须先确定 Vx、Vz 与转弯半径 R 的关系(这里以后轴中心为机器人旋转中心)。如下图 4-2-2 所示，假设机器人以速度 Vx、Vz 运动了 t 秒，则圆弧 O1O2 的长度为速度 Vx 对时间 t 的积分：O1O2= Vx*t；机器人姿态旋转了 θ，为速度 Vz 对时间 t 的积分： θ = Vz *t。由圆的性质易知，

转弯半径 $R = \frac{O_1O_2}{\theta} = \frac{V_X*t}{V_Z*t} = \frac{V_X}{V_Z}$ 式(4.2. 1)

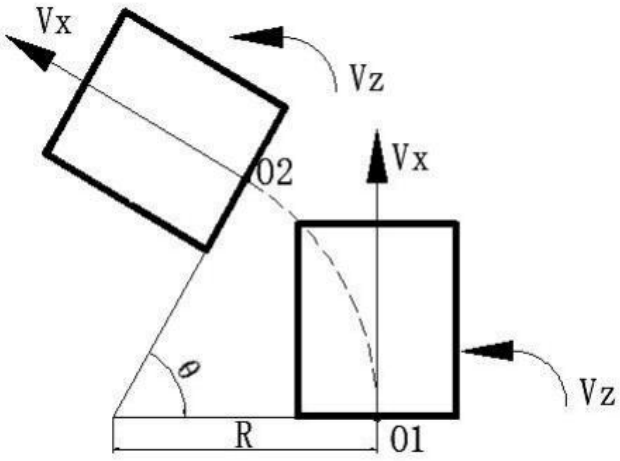


图 4-2-2 目标速度与转弯半径

现在已知 Vx、Vz 与转弯半径 $R = \frac{Vx}{Vz}$，接下来求左右后轮的速度 VL、VR 和左右前轮偏角 AngleL、AngleR。还需要知道的参数是轮距 W，轴距 H，如下图 4-2-3 所示。

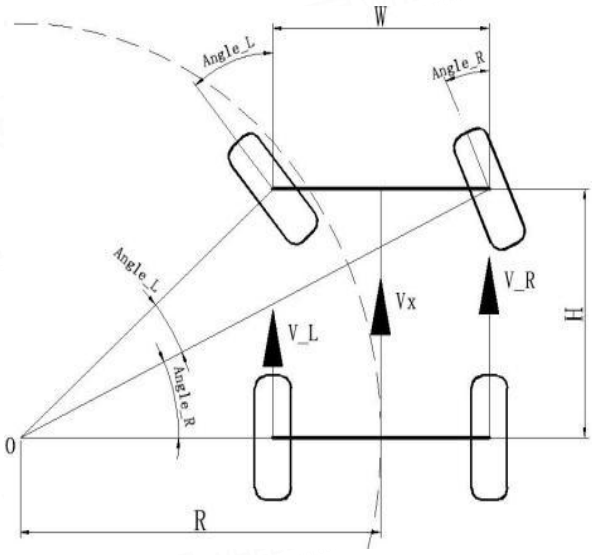


图 4-2-3 目标速度与轮子速度、偏角

如上图 4-2-3 所示，已知转向时机器人整体在绕 O 点做旋转运动，则易知左右后轮的速度 VL、VR：

$$VL = V_X \frac{R - 0.5w}{R} \quad 式(4.2.2)$$

$$VR = V_X \frac{R+0.5W}{R} \quad 式(4.2.3)$$

由几何关系可得左右前轮偏角 $Angle_L$ 、$Angle_R$：

$$AngleL = \tan^{-1}\left(\frac{H}{R-0.5W}\right) \quad 式(4.2.4)$$

$$AngleR = \tan^{-1}\left(\frac{H}{R+0.5W}\right) \quad 式(4.2.5)$$

$$转弯半径\ R = \frac{V_X}{V_Z} \quad 式(4.2.\ 1)$$

则运动学逆解公式已经求出来了，接下来求运动学正解：

$$VL + VR = Vx\frac{R-0.5W}{R} + Vx\frac{R+0.5W}{R}$$

$$VL + VR = 2Vx,则$$

$$Vx = \frac{VL+VR}{2} \quad 式（4.2.6）$$

Vx 求出来了，再求 Vz：

$$VR - VL = Vx\frac{R-0.5W}{R} - Vx\frac{R+0.5W}{R}$$

$$VR - VL = Vx\frac{W}{R}$$

$$R = Vx\frac{W}{VR-VL}$$

联合式（4.2.1）得

$$\frac{Vz}{Vx} = Vx\frac{W}{VR-VL}$$

$$Vz = \frac{VR-VL}{W} \quad 式（4.2.7）$$

总结，已知阿克曼式机器人轮距 W，轴距 H 机器人目标速度 VX、VZ 与左后轮速度 VL、右后轮速度 VR、左前轮偏角 AngleL、右前轮偏角 AngleR 的关系为：

逆解

$$R = \frac{Vx}{Vz} \quad 式(4.2.1)$$

$$VL = Vx\frac{R-0.5W}{R}$$ 式（4.2.2），$$VR=Vx\frac{R+0.5W}{R}$$ 式（4.2.3）

$$AngleL= \tan^{-1}Vx\frac{H}{R-0.5W}$$ 式（4.2.4）$$AngleR= \tan^{-1}Vx\frac{H}{R+0.5W}$$ 式（4.2.5）

② C 语言实现

C 语言实现代码如下：

```
void Get_Target_Encoder(float Vx,float Vz)
{
            float R, ratio=636.56, AngleR, Angle_Servo;
            //对于阿克曼小车 Vz 代表右前轮转向角度
             AngleR=Vz;
             R=Akm_axlespacing/tan(AngleR)-0.5f*Akm_wheelspacing;
             //前轮转向角度限幅(舵机控制前轮转向角度)，单位：rad
             AngleR=target_limit_float(AngleR,-0.49f,0.32f);
             //Inverse kinematics //运动学逆解
             if(AngleR!=0)
             {
                 MotorA.Target_Encoder = Vx*(R-0.081f)/R;
                 MotorB.Target_Encoder = Vx*(R+0.081f)/R;
             }
             else
             {
                 MotorA.Target_Encoder = Vx;
                 MotorB.Target_Encoder = Vx;
             }
}
```

对于阿克曼车，这里 Vz 代表的是转向的角度，以右转为正，左转为负。上述分析中，左转角度为正，所以这里和上述分析不矛盾。Akm_wheelspacing 和 Akm_axlespacing 分别是阿克曼车的轮距和轴距。

③ 如何控制前轮转角

通过运动学分析逆解，我们知道了实现目标速度 Vx、Vz，需要的左后轮速 度 VL、右后轮速度 VR、左前轮偏角 AngleL、右前轮偏角 AngleR。那么前轮偏角该如何实现呢？

下图 4-2-4 所示为基于曲柄摇杆机构的阿克曼转向结构，通过驱动舵机带动舵盘，舵盘再通过机械传动控制两个前轮进行转向。两个前轮偏角大小的差异则由机械设计决定，机械设计的要求为两个前轮的垂线与后轮轴线相交与一点。

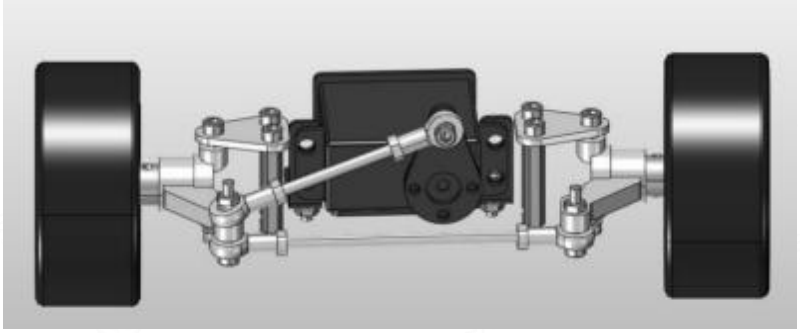


图 4-2-4 基于曲柄摇杆机构的阿克曼转向结构

由式(4.2. 1)、式(4.2.6)和式(4.2.7)我们知道每一组确定的目标速度 VX 、VZ，都有一组确定的 AngleL、AngleR，也即每一个 AngleL 都有一个对应的 AngleR，所以我们只需要知道舵盘的偏角与其中一个前轮对应的偏角关系即可。我们这里就只讨论舵盘偏角与右前轮的关系。

在这之前我们再讨论一下如何控制舵盘的偏角，我们的小车控制舵盘偏角都是使用舵机进行控制的，主要是因为舵机的控制需要的单片机资源很少，只需要一个定时器和一个 IO 引脚即可对舵机进行控制，实现舵盘的偏角控制。单片机是提供 PWM 控制舵机的，即不同大小的 PWM 与不同的舵机位置一一对应，然 后舵机带动舵盘进行转动，关于舵机控制大家可以到我们提供的资料包根目录下的【4. 电机控制基础视频教程】进行查看。

实际上大家也可以使用直流电机对舵盘偏角进行控制，但是这时候一个编码器获取电机的位置，进而负反馈控制达到控制舵盘偏角的目的。这时候需要的资源就需要两个定时器，一个用于控制电机，一个用于获取电机位置；5 个 IO 引脚，3 个用于控制电机，2 个用于获取电机位置。关于直流电机负反馈位置控制大家可以到我们提供的资料包根目录下的【4. 电机控制基础视频教程】进行查看。

此时我们知道使用舵机控制舵盘时，舵盘偏角与右前轮的关系，可以等价为舵机 PWM 控制值值与右前轮的关系，因为 PWM 与舵盘偏角是一一对应的关系 的。

那么因为这个曲柄摇杆的结构分析起来比较复杂，而舵盘偏角与前轮偏角是一一对应的，我们决定采用曲线拟合模型来近似的获得舵盘偏角与前轮偏角之间的关系公式。

下图 4-2-5 是我们的 ROS 教育机器人系列下的阿克曼机器人的舵盘偏角与前轮偏角关系的散点图。

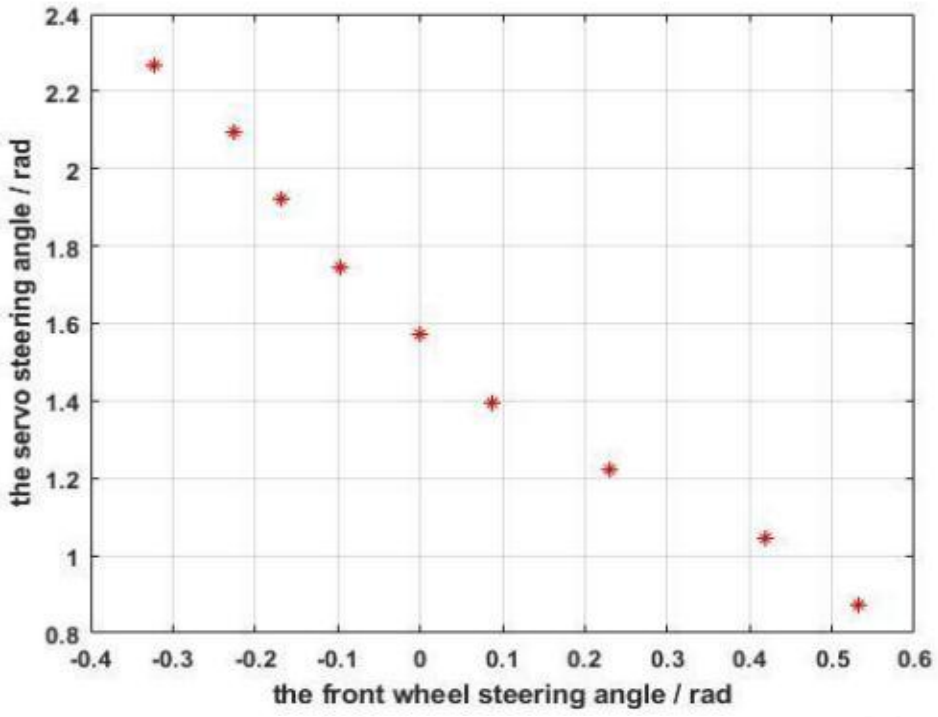


图 4-2-5 舵盘偏角与前轮偏角的散点图

通过观察，不难看出曲线的走势可以用一次曲线或者二次曲线来进行拟合。为了简单起见，可以调用 MATLAB 的曲线拟合工具箱（在 MATLAB 的命令行上输入 cftool 即可）查看两种曲线的拟合情况，如下图 4-2-6 到 4-2-9 所示。

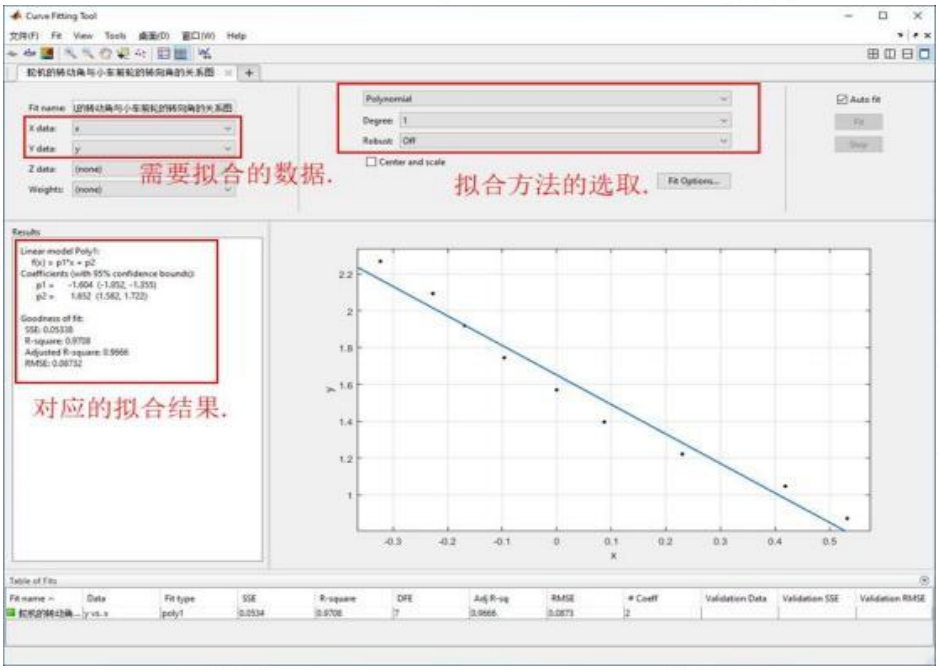


图 4-2-6 一次曲线的拟合情况

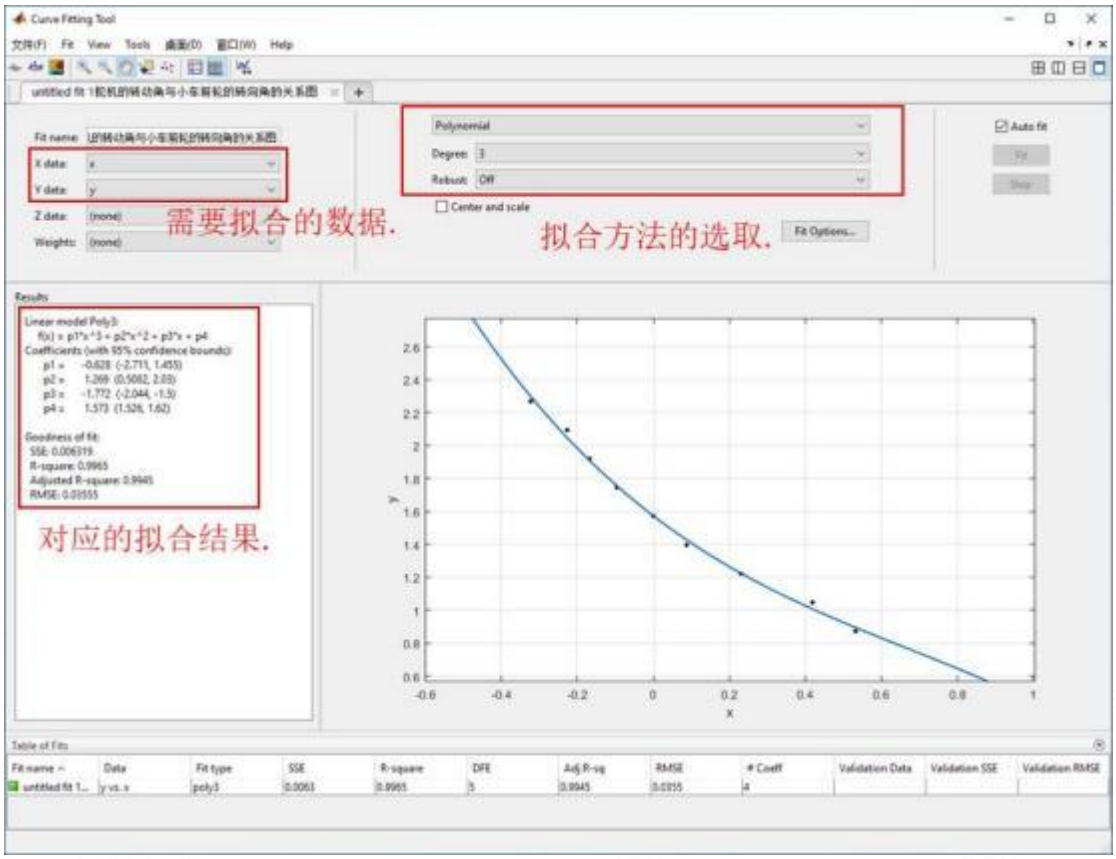


图 4-2-7 二次曲线的拟合情况

Linear model Poly1:
f(x) = p1*x + p2
Coefficients (with 95% confidence bounds):
p1 = -1.604 (-1.852, -1.355)
p2 = 1.652 (1.582, 1.722)

Goodness of fit:
SSE: 0.05338
R-square: 0.9708
Adjusted R-square: 0.9666
RMSE: 0.08732

图 4-2-8 一次曲线的拟合结果图

Linear model Poly3:
f(x) = p1*x^3 + p2*x^2 + p3*x + p4
Coefficients (with 95% confidence bounds):
p1 = -0.628 (-2.711, 1.455)
p2 = 1.269 (0.5082, 2.03)
p3 = -1.772 (-2.044, -1.5)
p4 = 1.573 (1.526, 1.62)

Goodness of fit:
SSE: 0.006319
R-square: 0.9965
Adjusted R-square: 0.9945
RMSE: 0.03555

图 4-2-9 二次曲线的拟合结果

- SSE：误差平方和(Sum of the Squared Error)，计算方法为

$$SSE = \sum_{i=1}^{n} (y_i - \hat{y}_i)^2$$

误差平方和越小说明曲线对观测值的拟合效果越好

- R^2 ：拟合优度(Goodness of Fit)，计算方法为

$$R^2 = 1 - \sum_{i=1}^{n} (y_i - \hat{y}_i)^2 \Big/ \sum_{i=1}^{n} (y_i - \bar{y}_i)^2$$

拟合优度越接近 1 说明曲线对观测值的拟合效果越好。

RMSE：均方根误差(Root Mean Squared Error)，计算方法为

$$RMSE = \sqrt{\frac{1}{n}\sum_{i=1}^{n} (y_i - \hat{y}_i)^2}$$

均方根误差的意义和误差平方和的意义一样，都是用以描述拟合值和观测值之间的误差大小。均方根误差越小说明曲线对观测值的拟合效果越好。因此，二次曲线对观测值的拟合效果比一次曲线对观测值的拟合效果稍好一点，这里选择二次曲线作为拟合曲线，对应的 C 语言代码如下所示。

```
    //Angle_Servo: 舵盘偏角, AngleR: 右前轮偏角
Angle_Servo = -0.628*pow(AngleR, 3) + 1.269*pow(AngleR, 2) - 1.772*AngleR + 1.573;
```

C 语言实现

以下是小车完整的 STM32 程序 C 语言实现。

由 X 轴目标速度和右前轮目标转角，求出左右电机的目标速度和控制舵机转动。这里的函数输入参数 Vz 代表右前轮目标转角。

```
void Get_Target_Encoder(float Vx,float Vz)
{
    float R, ratio=636.56, AngleR, Angle_Servo;

            // For Ackerman small car, Vz represents the front wheel steering Angle
            //对于阿克曼小车 Vz 代表右前轮转向角度
            AngleR=Vz;
            R=Akm_wheelspacing/tan(AngleR)-0.5f*Wheelspacing;
            // Front wheel steering Angle limit (front wheel steering Angle
controlled by steering engine), unit: rad
            //前轮转向角度限幅(舵机控制前轮转向角度), 单位: rad
            AngleR=target_limit_float(AngleR,-0.49f,0.32f);
            //Inverse kinematics //运动学逆解
            if(AngleR!=0)
            {
                MotorA.Target_Encoder = Vx*(R-0.081f)/R;
                MotorB.Target_Encoder = Vx*(R+0.081f)/R;
            }
            else
            {
                MotorA.Target_Encoder = Vx;
```

```
                MotorB.Target_Encoder = Vx;
            }

            // The PWM value of the servo controls the steering Angle of the front
wheel
            //舵机 PWM 值，舵机控制前轮转向角度
            Angle_Servo = -0.628f*pow(AngleR, 3) + 1.269f*pow(AngleR, 2) -
1.772f*AngleR + 1.573f;
            Servo_PWM=SERVO_INIT + (Angle_Servo - 1.572f)*ratio;
```

SERVO_INIT 即为舵机中值，即 1500；Vz 是转向的角度。

5.常用参数修改

本章内容主要列举一些用户需要修改的参数，方便用户查找和快速修改源码实现效果。

5.1 雷达避障参数修改

雷达避障可修改参数是避障范围的距离、前进的速度和转向速度，这些参数都是在 control.c 文件中的 void Lidar_Avoid(void)函数进行修改。

① 避障范围修改

避障范围需要修改函数中的 distance 变量，初始设定是差速车和小履带车在 350mm 内会进行避障，阿克曼车型在 400mm 内会进行避障，大履带车在 500mm 内会进行避障。若要修改，按照相应车型修改即可。

```
void Lidar_Avoid(void)
{
    int i = 0;
    u8 calculation_angle_cnt = 0;     //用于判断100个点中需要做避障的点
    float angle_sum = 0;              //粗略计算障碍物位于左或者右
    u8 distance_count = 0;            //距离小于某值的计数
    int distance = 350;               差速和小履带车
    if(Car_Num == Akm_Car)            阿卡曼车
        distance = 400;               //阿克曼车设定是400mm
    else if(Car_Num == Big_Tank_Car)  大履带车
        distance = 500;               //设定是500mm
```

图 5-1-1 避障范围修改

② 前进速度修改

主要修改 control.h 文件里面的参数，如图 5-1-2 所示。

```
#define Aovid_Speed 0.3          //避障速度
```

图 5-1-2 避障默认前进速度修改

③ 转向速度修改

主要修改里面的部分 Move_Z 参数，以小履带车为实例展示如何修改，其他车型参照即可。注意阿克曼车型只有转弯角度，无转向速度，修改后可能导致转向半径变大或变小，可能对避障效果有影响，不建议修改。

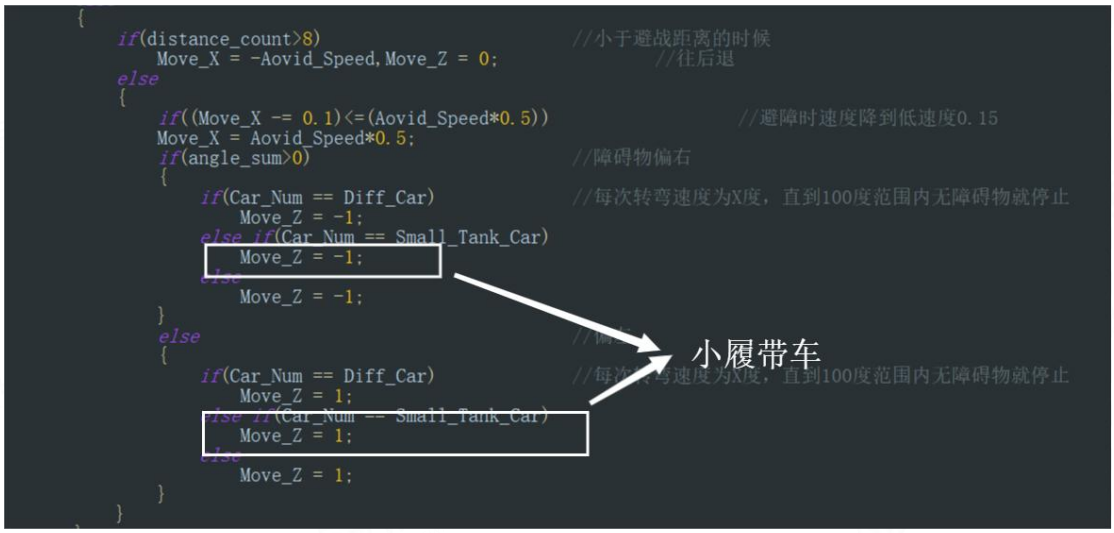


图 5-1-3 转向速度修改

5.2 遥控速度参数修改

小车可使用 PS2 手柄和蓝牙来进行遥控。如果使用蓝牙进行遥控，连接上蓝牙后，点击首页中的“加速”和“减速”可进行速度的增减；如果使用 PS2 手柄进行遥控，按下顶上的两个按键“L1”和“R1”来进行加减速。可以修改每次加速或减速的幅度，如图 5-2-1 所示。默认速度、最大和最小速度的修改如图 5-2-2、5-2-3 所示。这里的前后移动速度都是以 mm 为单位的，在具体函数时再转化为 m。上述参数修改在 control.h 文件中进行修改。



图 5-2-1 加减速幅度值

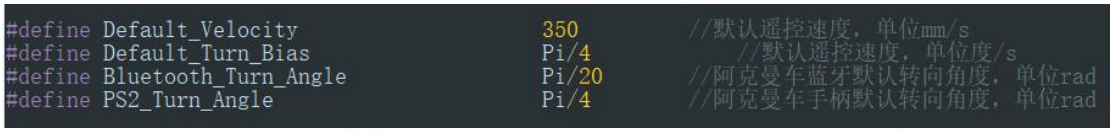


图 5-2-2 默认速度修改

```
//指示遥控控制的开关
#define RC_ON                       1
#define RC_OFF                      0
//遥控控制前后速度最大值
#define MAX_RC_Velocity             800
//遥控控制转向速度最大值
#define MAX_RC_Turn_Bias            1
//遥控控制前后速度最小值
#define MINI_RC_Velocity            210
//遥控控制转向速度最小值
#define MINI_RC_Turn_Velocity       Pi/20
```

图 5-2-3 最大和最小速度修改

5.3 雷达跟随参数修改

雷达跟随模式主要能修改的参数为跟随距离和跟随保持距离。

```
#define Follow_Distance 1000   //跟随距离
#define Keep_Follow_Distance 400   //跟随保持距离
```

图 5-3-1 雷达跟随参数修改

5.4 雷达走直线参数修改

雷达走直线主要能修改的参数就只有行驶的速度和参照物的方向。一般都是设置参照物的角度在我们小车的左上方或者右上方。

```
#define forward_velocity 0.25   //Move X速度
#define Along_Angle      77     //参照物的方向
```

图 5-4-1 雷达走直线的参数修改

5.5 电机或轮子更换后参数调整

若更换了电机或轮子或编码器，且规格和原来的不一样，则需要在代码中进行修改。

上述参数主要是在 control.h 头文件中进行更改。以下列出各个部件需要修改的参数。

① 修改轮子参数

轮子参数如下图 5-5-1 所示，主要是修改轮子的直径。

```
//车轮直径
#define Diff_Car_Wheel_diameter          0.0670f        //差速车和阿克曼车，单位m
#define Small_Tank_WheelDiameter         0.0430f        //小履带车
#define Big_Tank_WheelDiameter           0.0440f        //大履带车
```

图 5-5-1 轮子直径参数

② 电机参数修改

电机主要是需要修改减速比的参数，如图 5-5-2 所示。

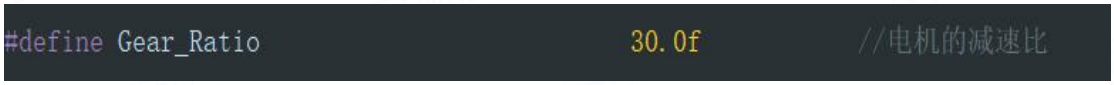


图 5-5-2 电机减速比修改

③ 编码器参数修改

编码器更改需要修改编码器的精度，如图 5-5-3 所示。



图 5-5-3 编码器精度修改

若更改了编码器和电机减速比，不仅需要更改相应的参数，还需要重新调整 PID 参数，这里给出一组仅修改编码器为 500 线的 PID 参数，其他精度的编码器需重新调参。PID 参数在 pid.c 文件进行修改。



图 5-5-4 PID 参数修改

5.6 扩展部分

主板预留了可拓展 IO，该部分针对这些拓展 IO 的使用进行讲解说明。需要注意的是，阿克曼车型因为需要用到舵机，已使用一路定时器资源，因此使用 AKM 车型时，超声波模块和航模遥控只有 3 个定时器通道可以使用，所以需要做一定的更改，以下教程针对非 AKM 车型。

① 四路超声波模块

超声波模块型号是 HC-SR04，最多可支持 4 路。HC-SR04 模块进行读取距离时主要是控制它的 Trig 和 Echo 引脚。给 Trig 引脚一个持续 10us 的高电平，再捕获 Echo 返回的持续的高电平时间，便可计算出测试到的距离。若要使用扩展代码，需要按下表 5-6-1 的说明接线。

表 5-6-1 超声波模块接线表

模块序号	Trig 引脚	Echo 引脚
1	PC15	PA1
2	PA12	PA2
3	PB13	PA3
4	PB12	PA11

接好模块后，需要在代码中做如下更改：

1. 需要在 main 函数里把注释掉的超声波初始化加上，然后在 capture.h 头文件中，加上 Distance_Capture 的宏定义，如下图 5-6-1 和 5-6-2 所示.

```
51    DEBUG_USART_Init();                 //调试串口设置，串口1，波特率115200
52    BLUETOOTH_USART_Init();             //蓝牙串口设置，串口3，波特率9600
53    LIDAR_USART_Init();                 //雷达串口设置，串口5，波特率115200
54    PS2_Init();                         //初始化PS2手柄接口
55    PS2_SetInit();                      //PS2手柄配置为模拟量模式
56    Car_Select_ADC_Init();              //车型选择ADC初始化
57    Voltage_ADC_Init();                 //用于测量电压
58  //Distance_Cap_Init(0XFFFF,72-1);     //用于测量超声波的距离，默认注释，具体看capture.h
59  //PWM_Cap_Init(0XFFFF,72-1);          //用于4路航模遥控初始化，默认注释，具体看capture.h   /*超声波和航模只能使用其一*/
60  //MPU6050_Init();                     //MPU6050初始化，默认注释，具体看MPU6050_Test.c的例子
61    while(!Key_Scan(KEY1_GPIO_PORT,KEY1_GPIO_PIN))      //按键选中小车型号，电位计可以选择
62    {
```

main函数里注释需去掉

图 5-6-1 超声波代码更改 1

```
41
42  //超声波和航模遥控的宏定义，只能使用其一
43  //选择不需要使用的注释掉
44  #define Distance_Capture
45  //define PWM_Capture
46
```

加上宏定义

图 5-6-2 超声波代码更改 2

2. 读取距离的时候，需调用 capture.c 中的 void Read_Distane(void)函数，可以放在 5ms 中断中进行调用。然后读取 Distance1,Distance2,Distance3,Distance4 这四个变量即可得到这四路超声波捕获的距离值。

② 航模遥控

使用单片机来获取航模遥控的发出的信号，实际上是捕获航模遥控接收机的 PWM 信号，遥控器中常说的几路通道机代表接收机能输出几路 PWM 信号。本项目最多支持四路航模遥控，但是只有捕获的功能，具体的一些计算和控制逻辑等需要您自行编写。

四路航模信号的捕获分别使用端口 PA1，PA2，PA3 和 PA11 进行捕获。需要使用这部分的扩展代码，需要做的修改为：

1. 在 main 函数里把注释掉的航模遥控初始化恢复，注意超声波的需要注释

掉。然后在 capture.h 头文件加上 PWM_Capture 宏定义。

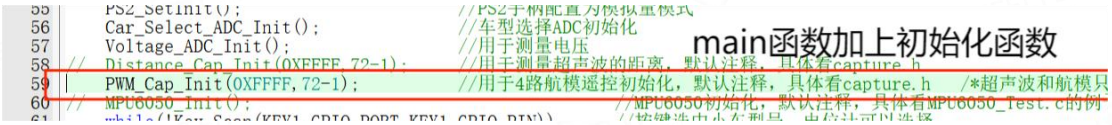


图 5-6-3 航模代码修改 1

2. 修改完成后需要自行写实现的控制代码



图 5-6-4 航模代码修改 2

③ MPU6050 模块

主板预留了 MPU6050 模块接口。若要使用 MPU6050 功能，插上 MPU6050 模块即可。

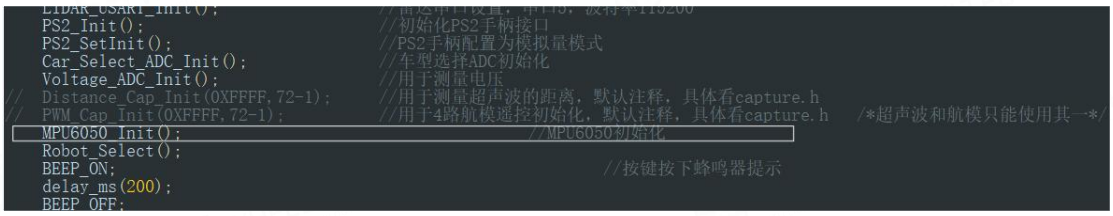


图 5-6-5 MPU6050 初始化

6. ROS 接口

L150 小车具备 ROS 机器人接口功能，对外提供 ROS 接口（串口控制与数据反馈），用户在原本 L150 小车基础上增加 ROS 主控与 ROS 传感器，即可组装成 ROS 机器人，如搭配我们公司的 ROS 机器人镜像，则不需要修改即可马上使用。通信协议分为发送数据与接收数据两个部分内容，详情请看下文。

6.1 ROS 接口控制输入协议说明

STM32 控制器板子上搭载了 CH9102(串口 1)一路串口通信接口，使用 CH9102(串口 1)和 ROS 端进行串口通信，以下为串口 1 接收数据说明。

接收数据采用中断接收的方式，接收的数据包括机器人产品信号、使能控制标志位、机器人三轴目标速度、数据校验位。

帧头帧尾默认是固定值；flag_stop 是机器人的使能控制位，默认发送使能；机器人三轴目标速度用于控制机器人运动，具体的接收内容见表 6-1。需要注意的是，表格中的数组号是上位机发送数据的数组号。

表 6-1 STM32 接收 ROS 发送过来的数据

数据内容	帧头(固定值 0X7B)	预留位	预留位	机器人 X 轴目标速度		机器人 Y 轴目标速度		机器人 Z 轴目标速度		数据校验位	帧尾(固定值 0X7D)
数据类型	Uint8	Uint8	Uint8	short		short		short		Uint8	Uint8
占用字节	1	1	1	2		2		1		1	1
数组号	1	2	3	4	5	6	7	8	9	10	11

注意：L150 系列小车不支持 Y 轴移动控制。

ROS 的控制模式是在 Normal（正常）模式下运行的，在正常模式的时候 STM32 控制器的串口 1 接收到数据，则进入 ROS 模式。设置前 10 秒不接收数据是因为消除机器人上电开机过程中发送过来的无用数据。10 秒的等待期后开始接收数据，首先要检测数据帧头，检测到数据帧头后才开始接收数据；在数据

接收完成后，再验证帧尾的数据校验位是否出错，数据校验位正确才使用数据。串口中断接收数据详细见 usartx.c 文件的 USART1_IRQHandler()串口中断函数。

6.2 ROS 接口默认发送的数据协议

ROS 和 STM32 控制器（运动底盘）之间通过串口实现通信，STM32 控制器使用的是串口 1，波特率是 115200。通信协议包括：STM32 控制器向 ROS 发送数据，ROS 向 STM32 控制器发送数据。打开 STM32 控制器的源码，关于串口通信的代码都在 STM32 控制器的 usartx.c 文件。

STM32 向 ROS 发送数据是按照 20hz 的频率执行。发送的数据包括：帧头帧尾、机器人使能标志位、机器人 XYZ 三轴速度、IMU 三轴加速度、三轴角速度、电池电压、数据校验位,详细发送的数据见下文的表 6-2-1。

发送数据的方式：将要发送的数据打包到一个数组中，数组的长度是 24 字节，使用串口逐位发送出去。因为串口一次只能发送一个 8 位（1 个字节）的数据，因此 2 个字节（short 型）的数据会拆分成高 8 位和低 8 位发送。

发送数据前对数据赋值的函数是“usartx.c”文件中的“data_transition( )”函数；发送数据的函数是“usartx.c”文件中的“USART1_SEND( )”函数。如果您需要更改发送的数据内容，直接更改 data_transition( )函数即可；如果您需要更改发送数据的长度，需要更改“buffer[ ]”这个数组长度的同时还需要“USART1_SEND( )”函数中 for 循环的次数，buffer[ ]这个数组长度修改 SEND_DATA_SIZE 即可，并且 ROS 端接收数据长度也要做相应的修改。

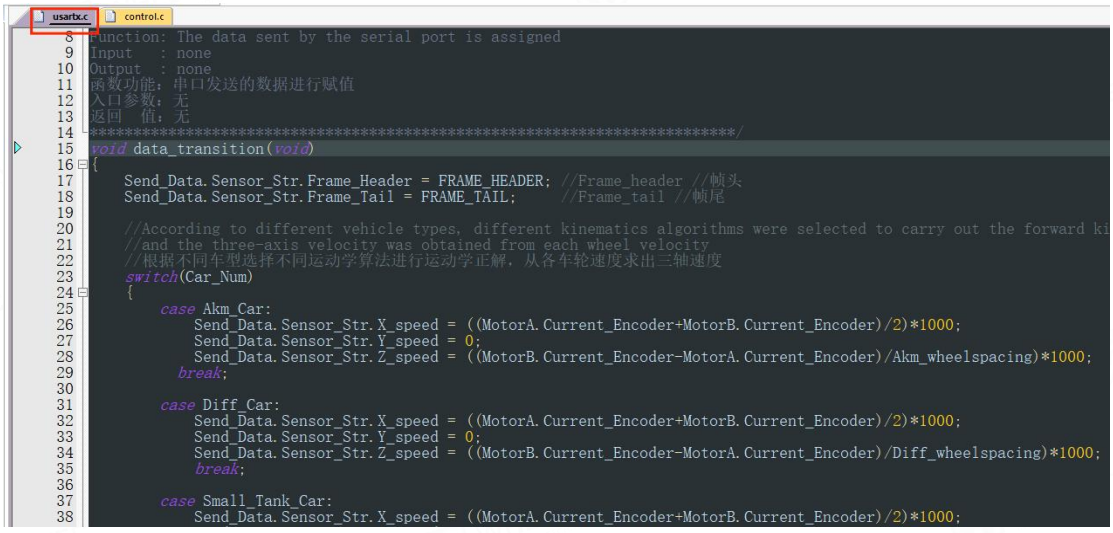


图 6-2-1 data_transition( )函数

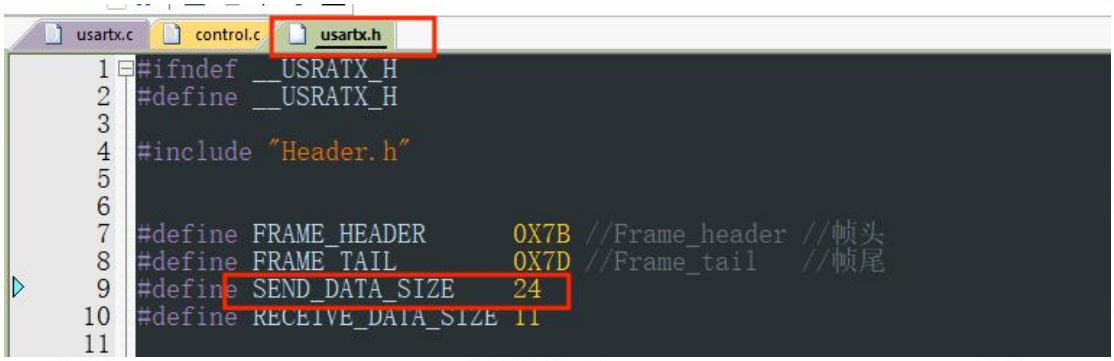


图 6-2-2 SEND_DATA_SIZE 位置

在发送的数据中需要说明的是，其中帧头是是固定值 0X7B，帧尾是固定值 0X7D，flag_stop 是电机的停止标志位（0 是使能、1 是失能）。数据校验位的计算方式是 BCC 校验（将全部数据位（包括帧头）异或），最终得到的结果就是数据校验位，数据校验位的计算过程见图 6-2-4。

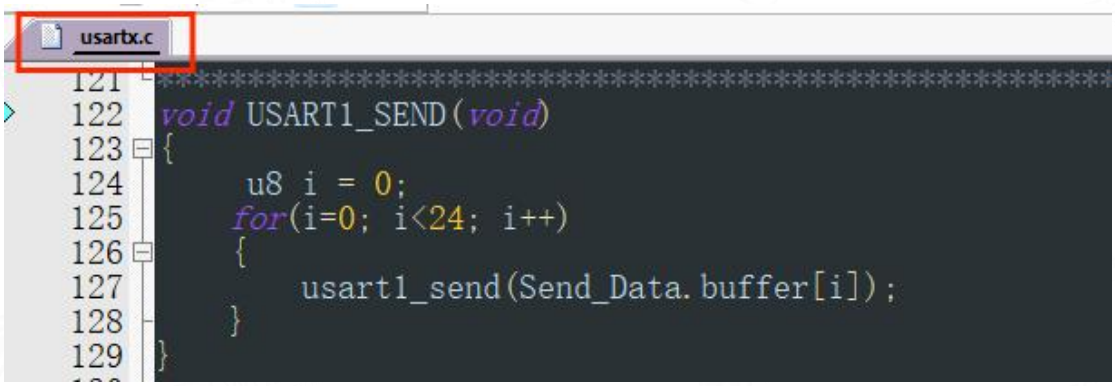


图 6-2-3 USART1_SEND( ) 函数

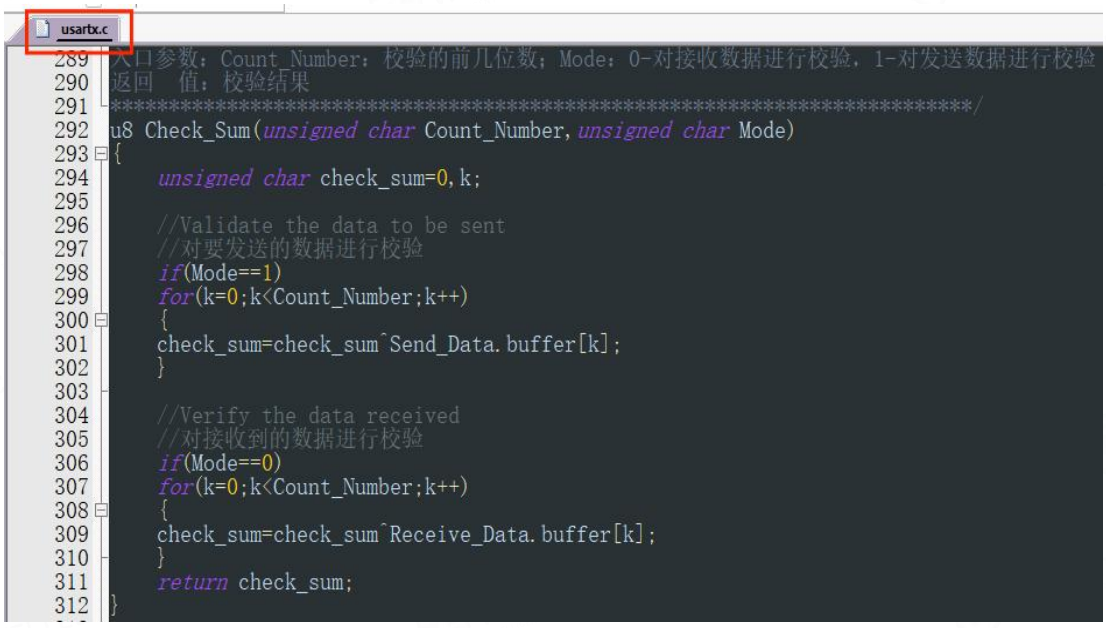


图 6-2-4 数据校验位计算函数

表 6-2-1 STM32 向 ROS 发送的数据

数据内容	帧头(固定值 0X7B)		flag_stop		机器人 X 轴速度		机器人 Y 轴速度		机器人 Z 轴速度		加速度计 X 轴加速度		加速度计 y 轴加速度	
数据类型	Uint8		Uint8		short		short		short		short		short	
占用字节	1		1		2		2		2		2		2	
数组号	1		2		3	4	5	6	7	8	9	10	11	12
数据内容	加速度计 z 轴加速度		角速度计 X 轴角速度		角速度计 Y 轴角速度		角速度计 Z 轴角速度		电池电压		数据校验位		帧尾(固定值 0X7D)	
数据类型	short		short		short		short		short		Uint8		Uint8	
占用字节	2		2		2		2		2		1		1	
数组号	13	14	15	16	17	18	19	20	21	22	23		24	

这里还有一个地方需要注意，机器人 XYZ 三轴速度、加速度计、角速度计以及电池电压的原始数据是浮点型的数据（float），因为浮点型数据使用串口传输不方便，所以这四个数据在发送之前先将浮点数放大一千倍（保留小数点后三位），再将放大后的浮点数强制转换成 short 型数据，最后在发送前将 short 型数据拆分成两个 8 位的数据。相应的，上位机端在接收到数据后，需要将接收到的数据两个 8 位数据合并后转换为 short 型，在缩小一千倍来进行单位的转换。

下面讲解一下如何将两个 8 位数据合并后转换为 short 型，即得到我们的实际速度等数据：我们的控制量单位是 mm/s(0.001m/s)，控制量方向由高 8 位数据的最高位决定。

示例 1：21 B6=0010 0001 1011 011 ，最高位为 0，正数，速度大小为 21

B6=(2*16+1)*256+(B*16+6)=(2*16+1)*256+(11*16+6)=8630(mm/s)；

示例 2：A1 2F=1010 0001 0010 1111，最高位为 1，负数，速度大小为 2^16(FF FF+1)-A1 2F=5E D0+1=(5*16+E)*256+(D*16+0)+1=24272(mm/s)。

下图为我们连接串口 1 后通过串口助手接收到的实际数据。

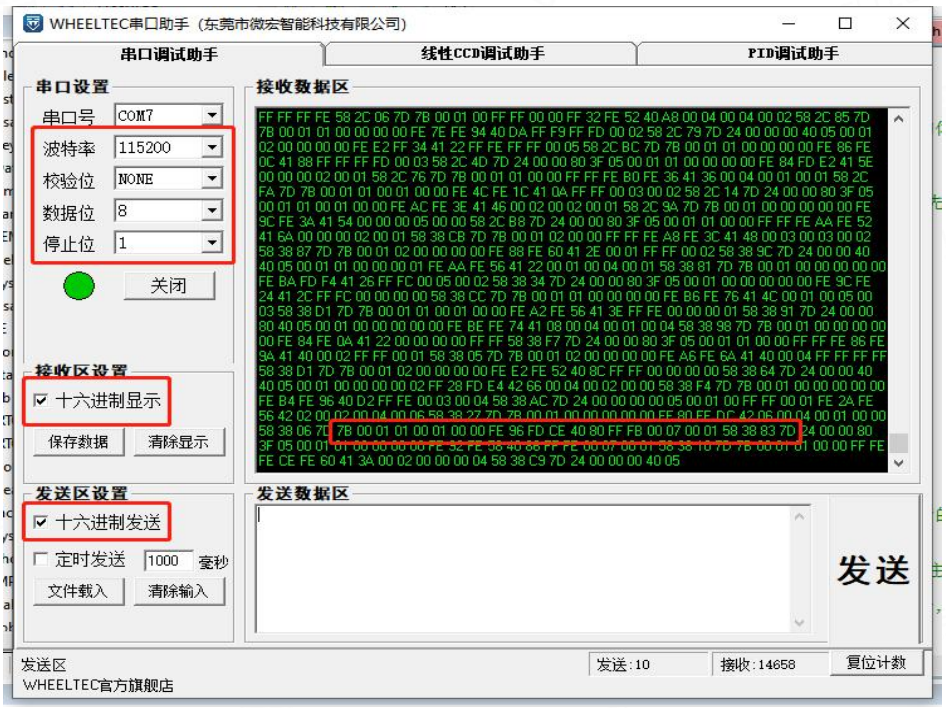


图 6-2-3 小车串口 1 发送的数据

下面对这接收到的 24 个字节数据转换一下：

第 1 个字节：0X7B，帧头；

第 2 个字节：0X00，电机处于非停止状态；

第 3、4 个字节：X 轴速度，高 8 位 0X01(16 进制)=0000 0001(2 进制)、低 8 位 0X01(16 进制)=0000 0001(2 进制)，最高位为 0，正数(前进)，速度大小为 1*256+1=257(mm/s)，该速度为小车对编码器数据进行计算得到的实际速度。

第 5、6 个字节：Y 轴速度，高 8 位 0X00(16 进制)=0000 0000(2 进制)、低 8 位 0X01(16 进制)=0000 0001(2 进制)，最高位为 0，正数(左移)，速度大小为 0*256+1=1(mm/s)，该速度为小车对编码器数据进行计算得到的实际速度。**(L150 系列默认为 0)**

第 7、8 个字节：Z 轴速度，高 8 位 0X00(16 进制)=0000 0000(2 进制)、低 8 位 0X00(16 进制)=0000 0000(2 进制)，最高位为 0，正数(逆时针旋转)，速度大小为 0*256+0=0(0.001rad/s)，该速度为小车对编码器数据进行计算得到的实

际速度。

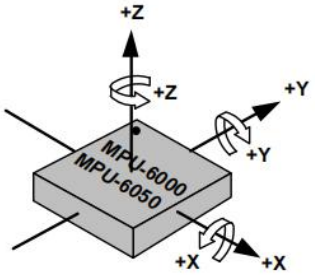


图 6-2-4 MPU6050 加速度计、角速度计三轴示意图

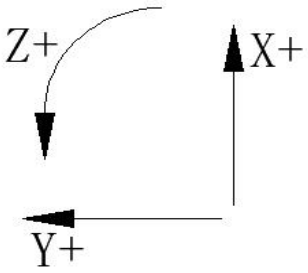


图 6-2-5 机器人 XYZ 三轴方向标定

（注意：L150 系列小车提供了 MPU6050 模块接口，插入即可使用。（如果没有接入 MPU6050 模块时，数据包的第九到第二十字节数据为 0））

接下来 12 个字节为三轴加速度计和角速度计的数据，注意这里 XYZ 三轴方向有所改变，X 轴正方向为向右，Y 轴正方向为向前，Z 轴正方向为上升。加速度计和角速度计的速度皆为绕 XYZ 三轴旋转的速度，如上图 6-2-4 所示。因为陀螺仪的 XY 轴方向与我们标定的机器人 XY 轴方向是有差别的，因此在 XY 轴的角速度和角速度数据在发送做了一个矫正的转换。

同时在 STM32 程序里面设置了 MPU6050 的加速度计量程为±2G=±2*9.8m/s^2=±19.6m/s^2，角速度计的量程为±500°/s=±8.72665rad/s。即例如读取计算得到 X 轴的加速度计数据为+32768，则代表此时机器人的前进加速度为+2G=+19.6m/s^2，32768/19.6=1672，则加速度数据转换为 m/s^2 的单位的比例参数为 1672。同理角速度数据转换为 rad/s 的单位的比例参数为 32768/8.72665=3755。

第 9、10 个字节：X 轴加速度，高 8 位 0XFE(16 进制)=1111 1110(2 进制)、低 8 位 0X96(16 进制)=1001 0110(2 进制)，最高位为 1，负数，大小为 2^16(FFFF+1)-FE 96=01 69=1*256+105=361，转换为加速度 361/1672=0.2159(m/s^2)。

第 11、12 个字节：Y 轴加速度，高 8 位 0XFD(16 进制)=1111 1101(2 进制)、低 8 位 0XCE(16 进制)=1100 1110(2 进制)，最高位为 1，负数，大小为 2^16(FFFF+1)-FD CE=02 31=2*256+49=561，转换为加速度 561/1672=0.3355(m/s^2)。

第 13、14 个字节：Z 轴加速度，高 8 位 0X40(16 进制)=0100 0000(2 进制)、

低 8 位 0X80(16 进制)=1000 0000(2 进制)，最高位为 0，正数，大小为 64*256+128=16512，转换为加速度 16512/1672=9.8756(m/s²)。

第 15、16 个字节：X 轴角速度，高 8 位 0XFF(16 进制)=1111 1111(2 进制)、低 8 位 0XFB(16 进制)=1111 1011(2 进制)，最高位为 1，负数，大小为 2^16(FF FF+1)-FF FB=00 04=0*256+4=4，转换为角速度 4/3753=0.0011(rad/s)。

第 17、18 个字节：Y 轴角速度，高 8 位 0X00(16 进制)=0000 0000(2 进制)、低 8 位 0X07(16 进制)=0000 0111(2 进制)，最高位为 0，正数，大小为 0*256+7=7，转换为角速度 7/3753=0.0019(rad/s)。

第 19、20 个字节：Z 轴角速度，高 8 位 0X00(16 进制)=0000 0000(2 进制)、低 8 位 0X01(16 进制)=0000 0001(2 进制)，最高位为 0，正数，大小为 0*256+1=1，转换为角速度 1/3753=0.0003(rad/s)。

第 21、22 个字节：电池电压，高 8 位 0X58(16 进制)=0101 1000(2 进制)、低 8 位 0X38(16 进制)=0011 1000(2 进制)，最高位为 0，正数，大小为 88*256+56=22584，电压大小为 22584mv(毫伏)。

第 23 字节：BCC 校验位(前 22 字节异或)，0X83=0X7B^0X00^0X01^0X01^0X00^0X01^0X00^0X00^0XFE^0X96^0XFD^0XCE^0X40^0X80^0XFF^0XFB^0X00^0X07^0X00^0X01^0X58^0X38。

（如果您想验证这个检验位的结果，可以使用一些网页在线平台来计算校验位）

第 24 个字节：0X7D，帧尾；

7.使用注意事项

7.1 关于小车使能开关

小车主控左上角红色开关为小车的使能开关，使能开关需要打到 ON 电机才受控制。如果小车电压超过 10V 但是不受控制时，应首先检查使能开关状态。

7.2 关于电磁传感器

在使用电磁传感器模块时，模块上两端的电感应始终以中间电感为对称轴两边对称，切忌随意改动两端电感的摆放位置，否则将影响巡线控制。

7.3 关于电机

使用的过程中要避免电机堵转，否则很容易烧毁主板。在小车没有完全测试通过之前，请把小车架起来，让电机悬空，避免误操作导致电机乱转引起不必要的损坏。

7.4 关于电池

显示屏上显示了电池的电压，电量低时（低于 10.0V）请及时充电。电池自带过放和过充保护。电池充电的时候请不要使用电池，电池充电接口如图 7-1 所示。



图 7-1 充电接口图示

8. 程序下载

程序下载可以使用串口一键下载，或使用 ST-Link 进行下载。

8.1 串口一键下载

串口一键下载步骤如下：

①硬件准备：小车与 TYPT-C 手机数据线

②软件准备：软件使用 MCUISP 烧录软件，在此之前先安装 USB 转 TTL 模块 CH9102 的驱动，附送的资料里面有驱动安装程序。驱动安装成功用数据线连接小车，打开电脑的设备管理器可以查看驱动是否安装成功。



图 8-1-1 驱动安装成功图示

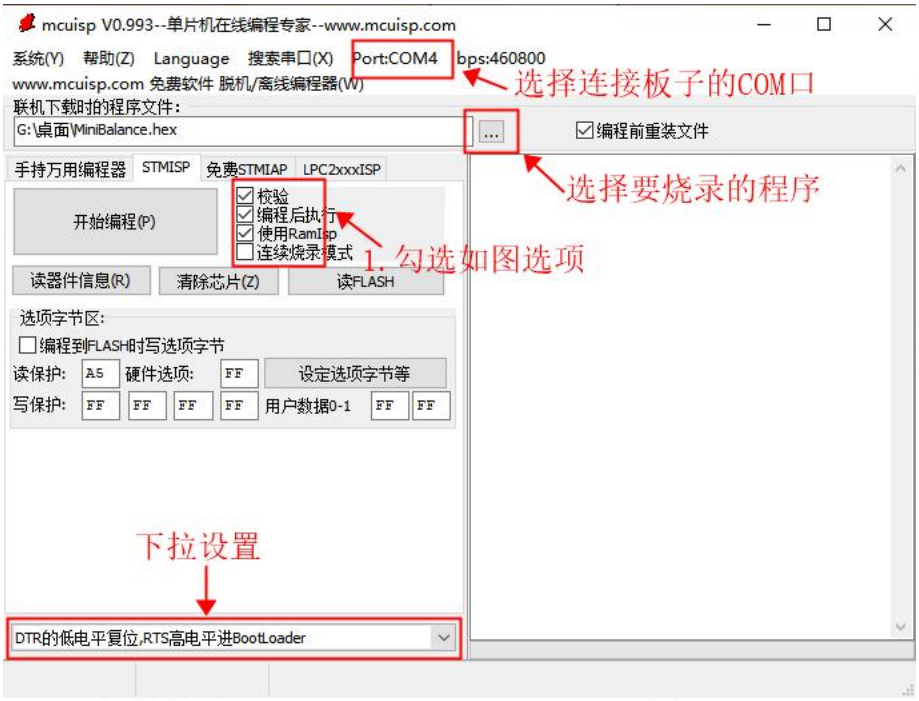


图 8-1-2 ISP 软件设置

从图 8-1-1 我们可以看到驱动已经安装成功，否则就会有红色感叹号。

③连接小车，打开附送资料的 MCUISP 软件，进行如图 8-1-2 的设置。程序文件在相应的工程文件下的 Output 文件夹里，后缀为.hex。

一切准备就绪，然后点击开始编程，程序就可以下载了。因为勾选了编程后执行，所以程序下载后会自动运行。

8.2 ST-Link 下载

使用 ST-Link 下载步骤如下：

1.硬件准备：ST-Link 下载器

2. 软件准备：Keil 编译器和 ST-Link 驱动程序。程序安装完成后连接 ST-Link 烧录器，打开电脑的设备管理器，点击通用串行总线设备会出现图 8-2-1 所示内容，表明安装成功。

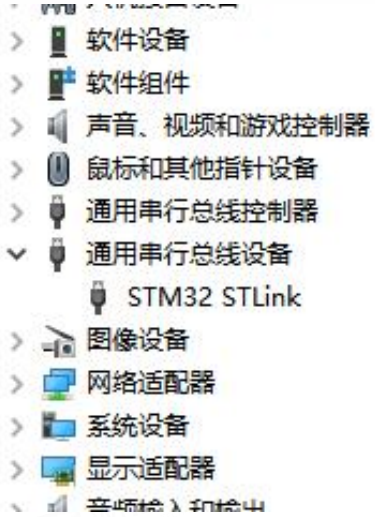


图 8-2-1 驱动成功安装图示

3.安装完成后，安装线序连接小车板子和 ST-Link，板子上的 ST-Link 连接接口在右上角排针的最后一排。



图 8-2-2 ST-Link 下载接口

Keil 软件需要进行设置，才能使用 ST-Link 烧录器。点击魔术棒-Debug 选项卡，进行图 8-2-3 和 8-2-4 所示设置。

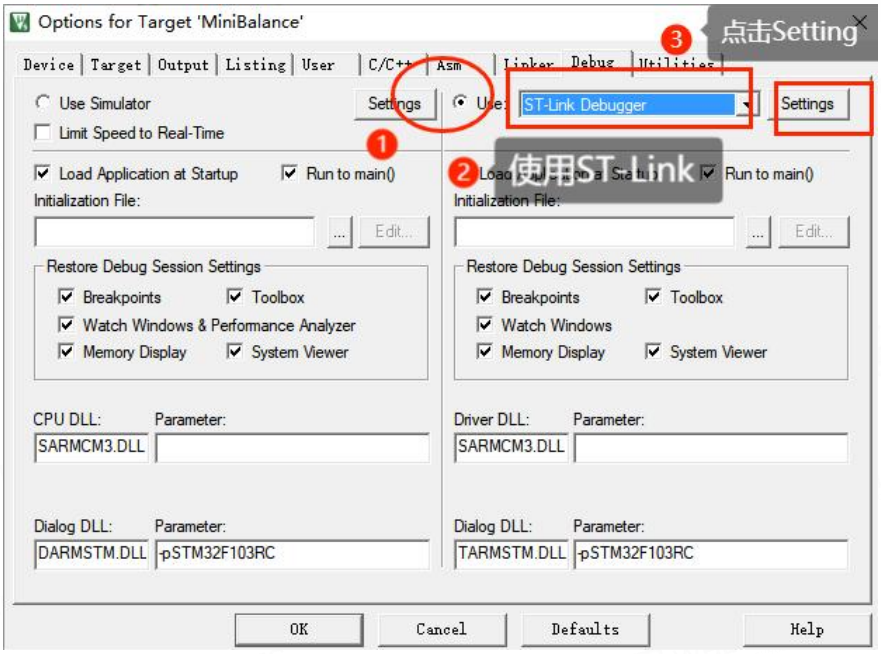


图 8-2-3 ST-Link 设置 1

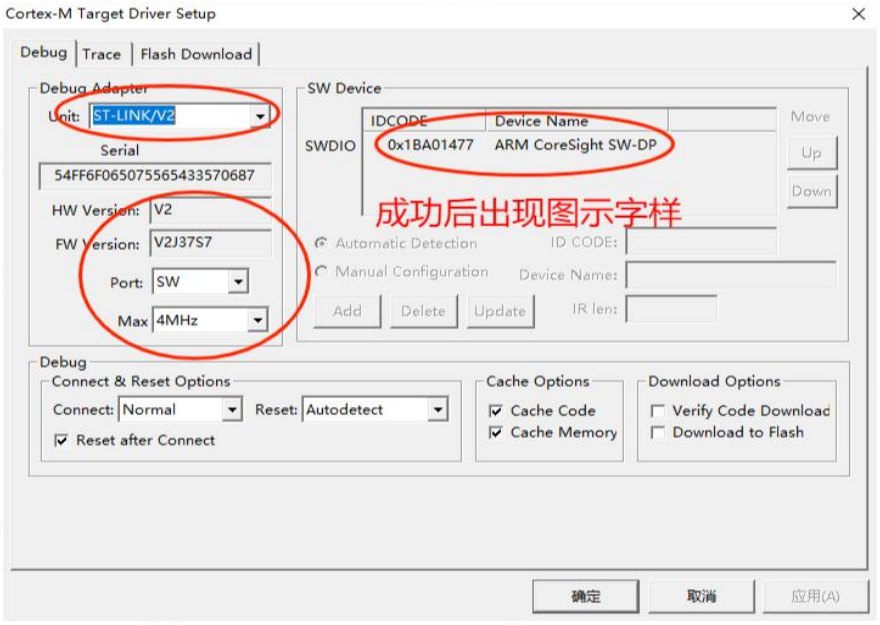


图 8-2-4 ST-Link 设置 2

设置完成后，重新编译，然后点击图 8-2-5 所示的下载按键即可下载到小车上

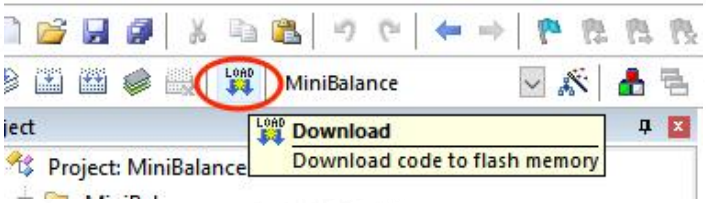


图 8-2-5 下载按键